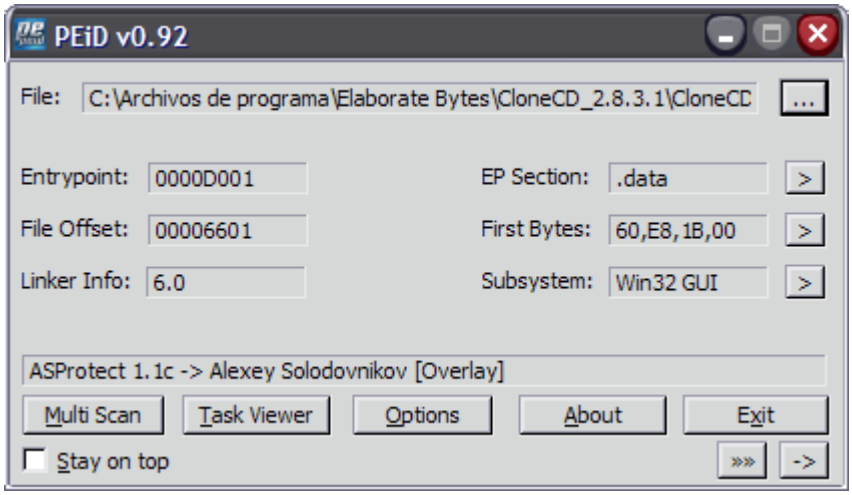
Programa:	CloneCD 2.8.3.1	
Protección:	AsProtect 1.22-1.23 y Trial 30 días	
Objetivos:	Desempaquetar el archivo y crackearlo	
URL descarga:	---	
Dificultad:	No fue facil, almenos para mi.	
Herramientas:	OllyDbg + PEiD + LordPE + PEditor + ImpRec + HexWorkShop.	
Cracker:	SyXe'05 [hAcK-c0d3d]	Fecha: 02-08-2005

Hola a tod@s...!!!

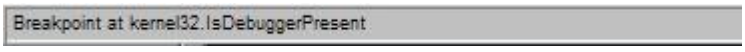
Aqui estamos una vez más, espero que estos tutos os estén ayudando por eso los hago para intentar ayudar el máximo posible a los que ahora comienzan, la verdad es que este tuto no lo iba a escribir, pero me encontré con una versión de CloneCD superior, en concreto la 3.2.3.1 y que tiene una protección anti-dump muy similar pero que aún no he conseguido pasar, jeje.. cosas que pasan, ya sabeis las cosas se complican y a veces dar con la solución no es fácil, normalmente en este mundo todo suele ser bastante complicado, sobretodo cuando uno es bastante inexperto ;)) pero bueno iremos progresando.

Vamos al ataque y menos paliqueo, bien abrimos el programa en el PEiD como hacemos siempre:

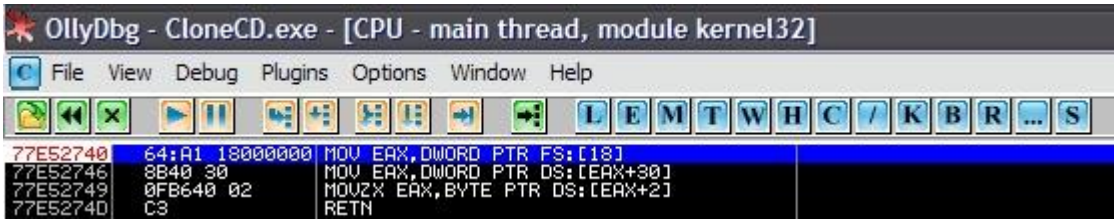


Un ASProtect de las versiones iniciales, ya ven que se trata de la versión 1.1c en peligro de extinción, jeje.. pero no nos fiemos de esto porque de verdad que hay versiones antiguas que plantean mayores retos que alguna más nueva sino ya vereis a lo que me refiero, os dire que contrariamente a como suelo hacer en este caso el programa ya lo tengo desempaquetado y funcionando, el que haya leido tutos mios sabra que ese no es mi estilo sino que yo escribo el tut a la par que voy trabajando en Olly con el programa original y voy contandolo todo tal cual me va sucediendo, mis ideas, planes, problemas que me encuentro, etc., pues pienso que eso es lo que realmente ayuda a los que se inician, no vale de nada decirle vete a tal direccion y cambia el salto, nopea la instruccion tal y cosas asi ya que lo unico que estamos haciendo es darle un crack y no ayudandole a que aprenda a crackear, cuando bajo mi punto de vista los tuts deben estar orientados a enseñar a crackear o como lo querais llamar. Como subrayo al principio del tut estoy escribiendo este porque solo tengo el archivo dumpeado y algunas notas claro, a partir de aqui voy a dumpear de nuevo y todo, reparar los antidumps, etc., del archivo original y asi entender un poco mas esta protección para después enfrentarme de nuevo al otro Clone que de momento se resiste ¿ok?

Carguemos el ejecutable original ***CloneCD***.exe en ***Olly***, para los que no tenga el plugin del ***IsDebuggerPresent*** imagino que sabran ocultar el debugger, sino esto es bien facil, estando parados en ell ***EP*** del programa ***:40D001*** vamos al ***CommandBar*** y ponemos ***BP IsDebuggerPresent*** y aceptamos pulsando ***Enter***, el BP quedara puesto, si al lado aparece el texto ***Unknown Identifier*** probablemente os habreis saltado una letra al escribirlo, repasar esto y lo poneis bien hasta que no diga nada, ya estara puesto el BPX en la api, ahora dejamos correr el programa con ***F9*** tras superar 13 excepciones en mi caso nos detenemos por el BreakPoint como dice la barra de estado de Olly:



Veamos un poco como funciona esta api, aunque realmente hay poco que ver, veamosla:

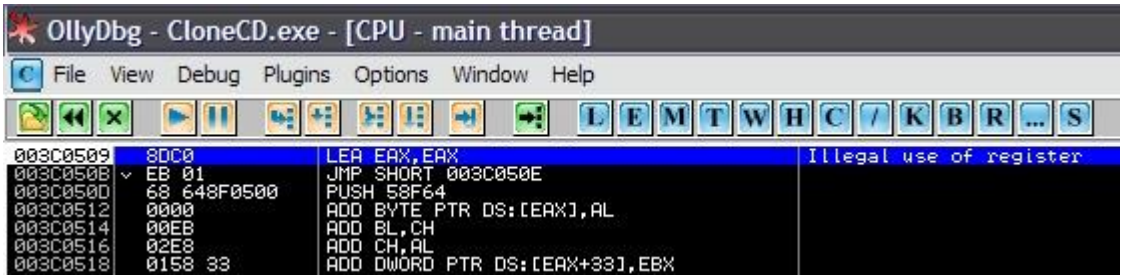


Como vemos la api no es gran cosa, son solo cuatro lineas, lo interesante esta en la instruccion:

:77E52749 0FB640 02 MOVZX EAX,BYTE PTR DS:[EAX+2]

donde EAX toma el valor de ***[EAX+2]***, si vamos traceando hasta llegar a ella y hacemos un ***Follow in Dump -> Memory Address*** descubrimos que ***[EAX+2]*** representa ***[7FFDF002]*** y este es el flag que indica si estamos trabajando bajo un debugger o no, en vuestra máquina puede variar este valor pero el proceso es totalmente el mismo, lo que de debemos hacer es cambiar el 1 que allí habrá por un 0, si tenemos el plugin del IsDebugger... activado lógicamente nos podemos evitar este paso, pero lo explico porque no creo que esté de más aprender,, ya digo que el tuto no va a ser ni muy largo ni muy corto, así que comodáos, abrid una cerveza o lo que queráis con una pizza o lo que más os guste, aqui venimos a aprender cosas nuevas y no tan nuevas porque todo lo que yo aqui explique con 5 minutos de visita a la pagina de CrackslatinoS o a cualquier FTP de Ricardo Narvaja, [juan Jose], etc.. obtendréis info suficiente para derrotar este paker, de hecho yo aprendo de sus tuts, jeje.. está claro hay que aprovechar lo que otros escriben y luego porsupuesto escribir lo que aprendes para que otros lo aprendan, como vemos es como una cadena que no se debe romper si todos hacemos lo mismo, aprender – escribir ;DDD asi que cuando aprendáis algo lo mejor que podeis hacer es escribir el tuto para compartir con los demás lo aprendido, bueno esto más que un tuto se está pareciendo a una campaña electoral, pasemos a lo que hemos venido..

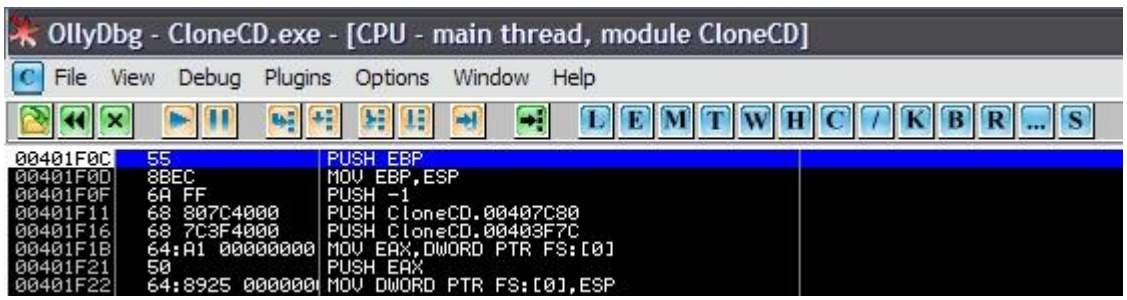
Ya superado el truquito del antidebug (bastante pobre por cierto) seguimos superando excpciones con Shift+F9 hasta que el programa cargue completamente, en pantalla habrá quedado la última excepción que deberemos anotar, si no quedó nopas nada picamos la taecla '-' del teclado numérico y aparecerá, es este caso es esta:



Este dato nos será de utilidad después para llegar al OEP cuando reparemos la IAT, lo anotamos entonces:

<u>Última excepción</u>: 3C0509

Sigamos, reiniciamos con Ctrl+F9, y al llegar a la última excepción vamos a la ventana de memoria y ponemos un BPM On Acces en la primera sección que es la que hay justo después del PE header y comienza en :401000 cuando el imagebase es 400000 claro, no pongo imagen porque con esto no habra problema suopngo, ahora superamos la excepción y caemos en el OEP del programa:



Este es el ***OEP***, el programa tiene pinta de estar compilado en C++ sin msvcrt, ***PEiD*** nos dice ***Visual C++ 6.0***, perofecto ahora nos anotamos el OEP como hicimos con la última excepción:

OEP: 401F0C

Este dato nos hará falta también, ahora busquemos la IAT, no hay que bajar mucho para ver esta instrucción:

:00401F32 FF15 C8704000 CALL DWORD PTR DS:[4070C8]

Es un salto típico a la ***IAT***, como es el primer call de este tipo y sabiendo el compilador utilizado estamos casi seguros de que debería apuntar a ***GetVersion*** pero no nos adelantemos todavís, pinchamos con el botón derecho y hacemos un ***Follow in Dump -> Memory Address*** para caer en plena tabla (IAT), ahora busquemos el inicio de la misma subiendo hacia arriba:

```
Address   Hex dump                                          ASCII
00407000  D8 17 DA 77 B1 63 DB 77 D6 27 DA 77 E2 68 DB 77  ï‡ rw▒c▄wí' rwôh▄w
00407010  10 24 DA 77 9A 22 DA 77 00 00 00 00 34 03 90 00  ►$ rwÜ" rw....4♥É.
00407020  58 11 90 00 E0 02 90 00 64 11 90 00 08 C2 3B 00  X◄É.☺☻É.d◄É.◘┬;.
00407030  7C 11 90 00 8C 11 90 00 9C 11 90 00 A8 11 90 00  |◄É.î◄É.£◄É.¿◄É.
00407040  BC 11 90 00 CC 11 90 00 DC 11 90 00 EC 11 90 00  ╝◄É.╠◄É.▄◄É.ý◄É.
00407050  FC 11 90 00 1C 12 90 00 2C 12 90 00 5C 12 90 00  ³◄É.∟↕É.,↕É.\↕É.
00407060  6C 12 90 00 7C 12 90 00 90 12 90 00 AC 12 90 00  l↕É.|↕É.É↕É.¼↕É.
00407070  C4 12 90 00 D4 12 90 00 E0 12 90 00 F4 12 90 00  ─↕É.É↕É.Ó↕É.¶↕É.
00407080  00 13 90 00 0C 13 90 00 18 13 90 00 24 13 90 00  .‼É..‼É.↑‼É.$‼É.
```

En este caso no planteaba ningún problema ya que es el inicio de la sección ***.rdata*** suponemos ya que ***ASProtect*** les borra el nombre, o bien ***.idata*** aunque el nombre es lo de menos, ya tenemos el primer valor, ahora busquemos el final bajando hacia abajo y de la misma manera:

```
Address   Hex dump                                          ASCII
004070D0  40 14 90 00 50 14 90 00 5C 14 90 00 6C 14 90 00  @¶É.P¶É.\¶É.l¶É.
004070E0  78 14 90 00 8C 14 90 00 98 14 90 00 A4 14 90 00  x¶É.î¶É.ÿ¶É.Ñ¶É.
004070F0  B8 14 90 00 CC 14 90 00 DC 14 90 00 EC 14 90 00  ╕¶É.╠¶É.▄¶É.ý¶É.
00407100  FC 14 90 00 08 15 90 00 18 15 90 00 30 15 90 00  ³¶É.◘§É.↑§É.0§É.
00407110  00 00 00 00 3C 15 90 00 48 15 90 00 58 15 90 00  ....<§É.H§É.X§É.
00407120  64 15 90 00 70 15 90 00 7C 15 90 00 8C 15 90 00  d§É.p§É.|§É.î§É.
00407130  98 15 90 00 A4 15 90 00 B4 15 90 00 C4 15 90 00  ÿ§É.Ñ§É.┤§É.─§É.
00407140  D4 15 90 00 E4 15 90 00 F0 15 90 00 00 00 00 00  É§É.ä§É.-§É.....
00407150  00 00 21 10 42 20 63 30 84 40 A5 50 C6 60 E7 70  ..!►B c0ä@Ñ╞Pä'╞p
```

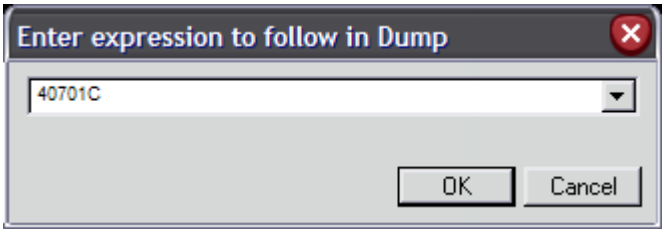
Parece una iat bien pequeña, ademas me mosquea bastante porque en su directorio de instalacion veo archivos como ***ElbyCDIO.dll*** (cuidado que tiene la pripiedad de estar oculto) y los saltos a las ***dll*** no las suele redirigir y no veo ninguno ¿? he bajado bastante pero no he encontrado nada, espero que esto no nos de problemas, de momento nos quedamos con estos datos:

IAT => Inicio: 407000
Final: 407148
Largo: 148

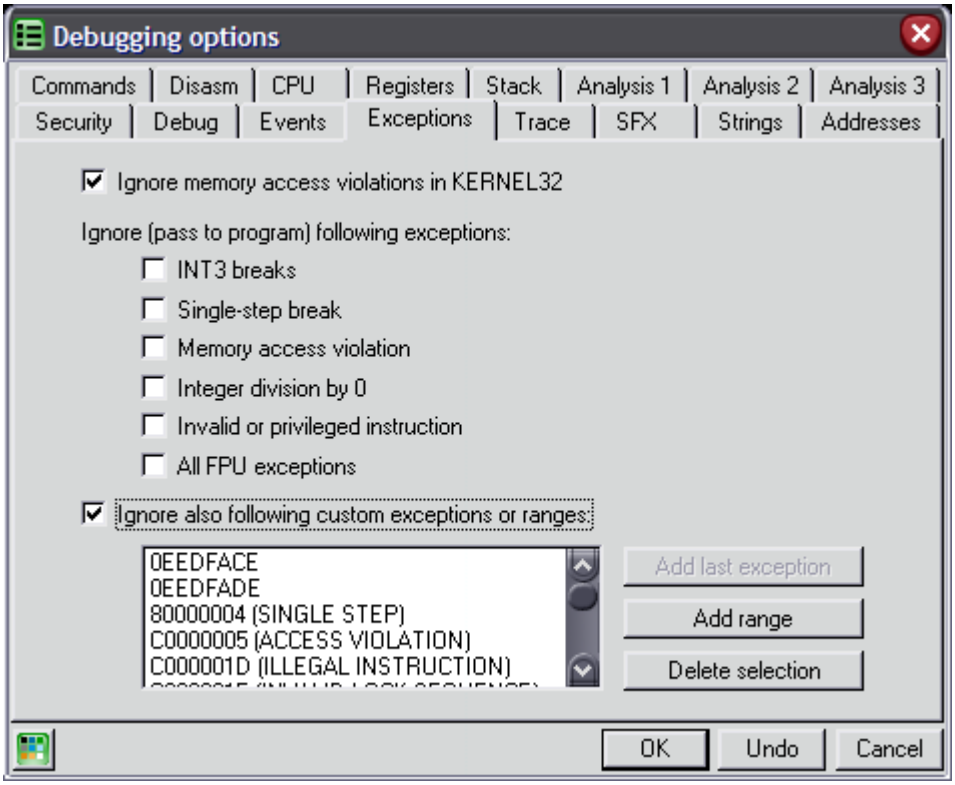
Si tenemos dudas si un puntero pertenece o no a la iat seleccionamos los bytes y pulsamos ***Ctrl+R***, aparecerá una ventanita con las referencias, si son tipo ***jmp dword ptr [xxxxxxxx]*** ó ***call dword ptr [xxxxxxxx]*** está claro que ***SI*** forma parte de la iat, en otro caso sabemos que no. Ahora vamos a buscar el call semi-mágico, esto se hace así: Seleccionamos un puntero redirigido de la tabla y anotamos su valor final, por ejemplo este:

[40701C]=900334

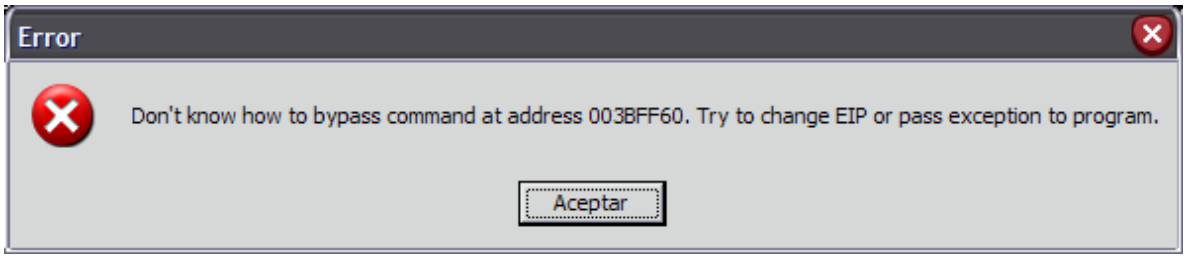
Reiniciamos con Ctrl+F2, estando en el EP vamos a esa dirección:



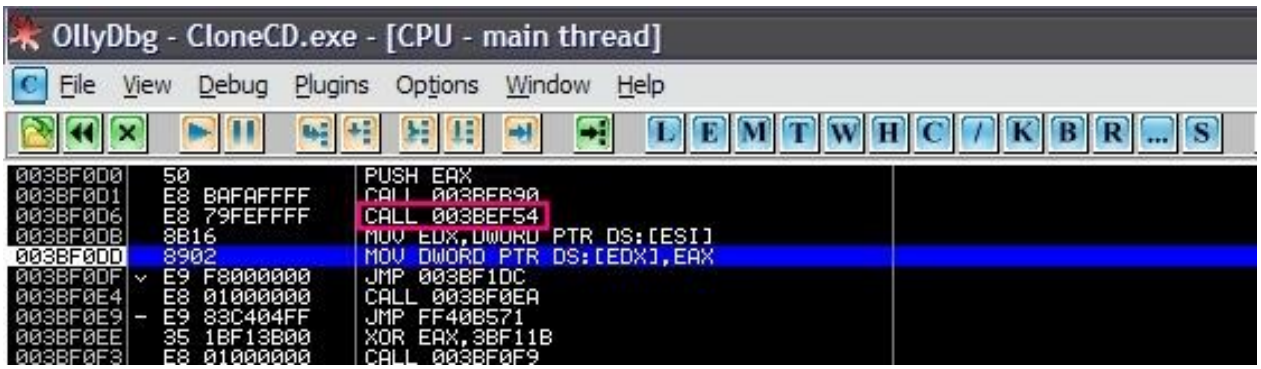
Pulsamos ***OK*** y caemos el el puntero, le ponemos un ***BPM On Write*** y dejamos correr la aplicación, para que no nos molesten desactivamos las excepciones, mostraré una imágen, después cuando me refiera a ***activar/desactivar*** las ***excepciones*** recordad este cuadro:



Lo que hice fue tildar la patilla ***Ignore also following custom exceptions or ranges*** que significa que queremos que ignore las excepciones que están allí especificadas, si no hay ninguna añadida pues entonces tildamos todas, fijaos que en ***Ignore memory acces violations in KERNEL32*** no debemos tocar, hay que dejarla como está en la imagen, es decir, siempre tildada. Ahora damos ***RUN*** ó ***F9*** para que corra, parará tres o cuatro veces antes de llegar al punto clave, pero como sabemos que es nuesrta parada? recordad que os dije que anotaramos un puntero redirigido y su valor final? pues ahí está la respuesta, estaremos en la zona buscada cuando tome al valor final, jeje.. el resto de paradas las ignoramos simplemente pulsando ***F9***, veremos el siguiente mensaje un par de veces:



Por esto no nos preocupemos, simplemente lo aceptamos y luego superamos la excepción son ***Shift+F9*** y seguimos normalmente, si vuelve a aparecer pues nada hacemos lo mismo, al fina encontraremos el call buscado:

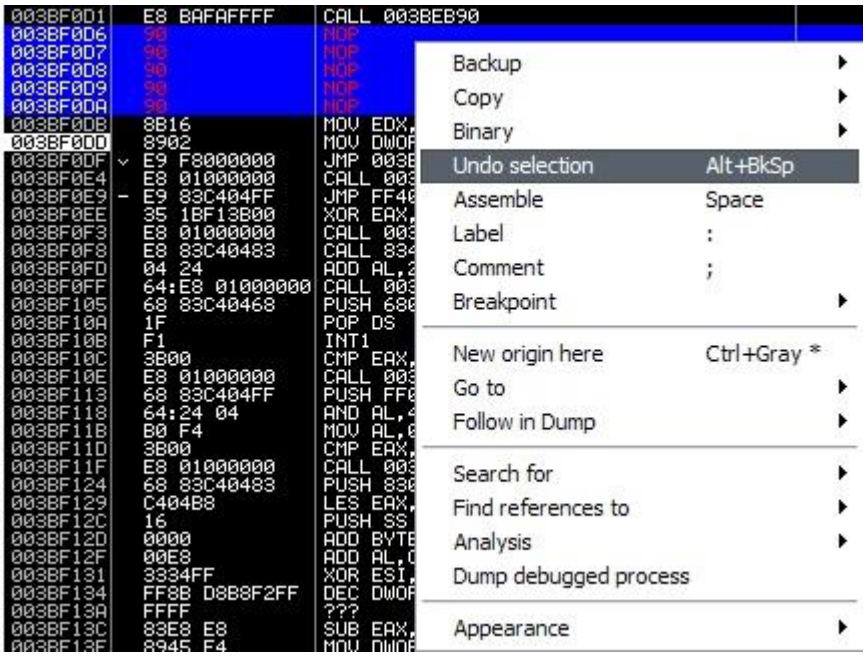


Nuestro call es el que remarqué, es el segundo de ellos e inconfundible, primeramente porque siempre viene un ***PUSH EAX*** y luego los dos calls seguidos, ademas si ponemos un ***HBPX*** sobre el primer call veremos que llegamos con un valor api en ***EAX*** y al pasar sobre el segundo call este valor es reemplazado por uno de ASProtect lo que indica con total seguridad que es el call semi-mágico ;DDD

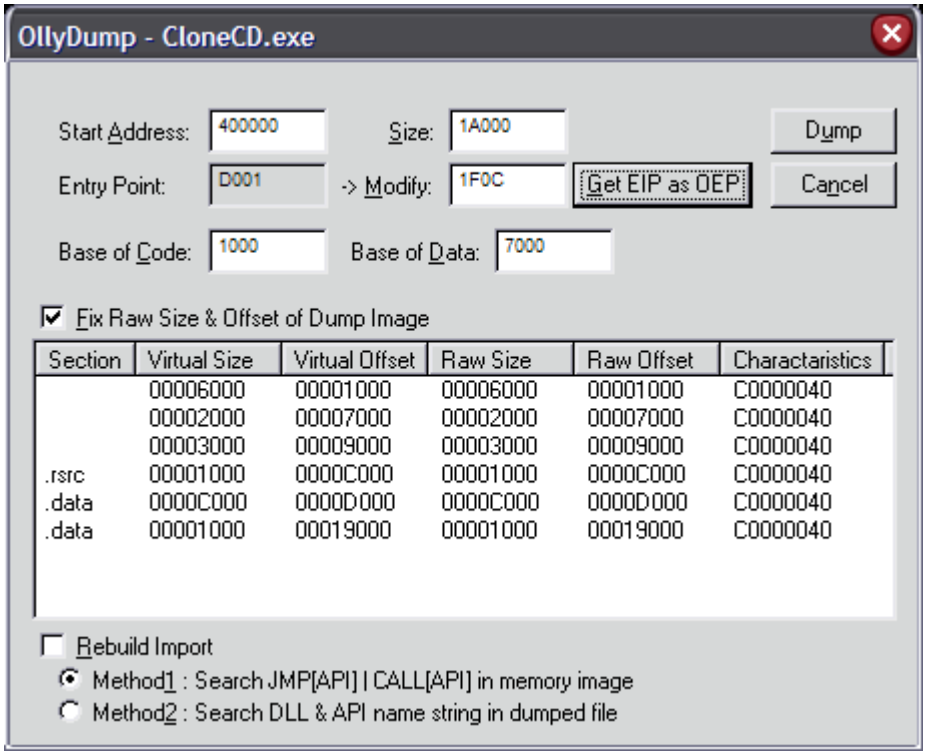
Call semi-mágico: 3BF0D6

Ahora lo que se suele hacer es reiniciar comosiempre con ***Ctrl+F2*** llegar a la primera excepción y desde allí llegar a la dirección del call con ***Ctrl+G***, ahora le ponemos un ***BPX*** encima, ***Olly*** nos avisará con una mensaje, nada pasamos de él, luego desactivamos las excepciones para que no pare en ellas y damos RUN, cuando estemos sobre el call quitamos el BPX con F2 nuevamente y lo nopeamos:

No se ve muy bien pero las instrucciones en rojo son los nops, ahora antes de pulsar ***F9*** vamos al último puntero de la tabla en la ventana dump que era ***:407148*** y le ponemos un ***BPX on Memory Write*** para que ***Olly*** pare cuando se escriba sobre él, esto lo hacemos para dejar el call en su lugar, si cuando Olly pare no tenemos los nops a la vista nos ponemos en la ventana de codigo y vamos pulsando la tecla '-' hasta que aparezca en pantalla, normalmente a la primera o segunda vez ya aparecera, seleccionamos todos los nops y con el boton derecho pinchamos y elegimos ***Undo Selection***, el call reaparecera en su sitio, para ASProtect nada habra pasado, jeje.. si no hacemos esto no llegaremos al ***OEP*** ya que el chequeo ***CRC32*** nos tiraria fuera seguramente, solo he visto un ASProtect que no te tire o sea que lo tienen casi todos:



Ahora activaremos nuevamente las excepciones com ya antes hemos explicado para que Olly se detenga en ellas, asi nos detendremos en la ultima excepcion que como sabemos ocurre en la direccion ***:3C0509*** ya que habremos anotado antes este valor, después pulsamos ***Alt+M*** y nos metemos en la ventana de memoria, desde allí ponemos un ***BPM*** en la primera seccion mediante la opción ***Set memory breakpoint on acces***, ahora pulsamos ***Shift+F9*** ya que estaremos detenidos en una excepcion para caer de pleno en el OEP del programa, aqui ahora ya podemos dumpear, utilizaremos el plugin ***OllyDump*** del propio Olly asi configurado:

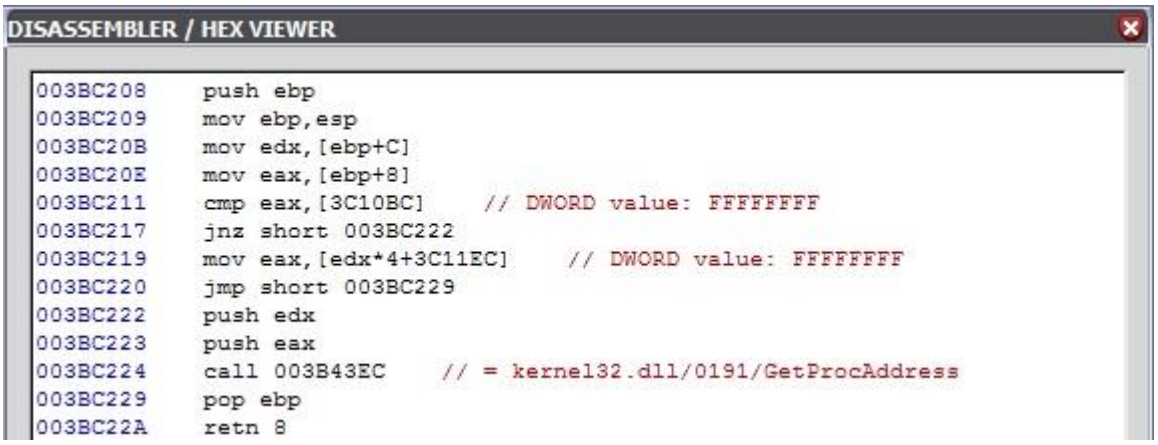


Recordad destildar la opción ***Rebuild Import*** ya que no reparará la iat y esto lo haremos nosotros luego, si pulsaremos en ***Get EIP as OEP*** para que nos ponga el ***OEP*** correcto y luego en ***Dump***, le damos un nombre y listo..!! ya tenemos el archivo dumpeado, pero todavía queda mucho trabajo, vayamos al Import Recostructor (***ImpRec*** a partir de ahora) seleccionamos el proceso de ***CloneCD.exe*** y le damos estos valores:

Tendremos siempre en cuenta que trabajamos con offsets, picamos en Get Imports y nos informa de que solo quedo unpuntero opr reparar, sus datos son estos como alli podemos ver:

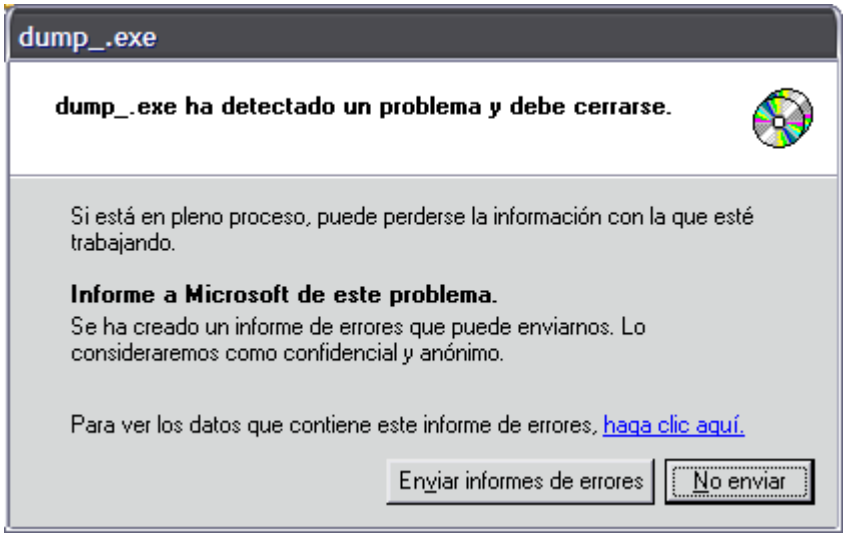
<u>Entrada 1</u>:

Iat rva: 702C
Apunta a: 3BC208

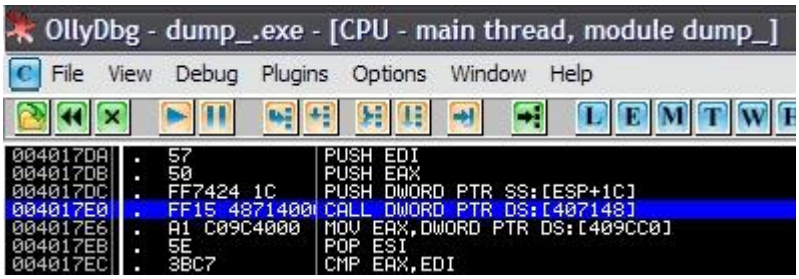


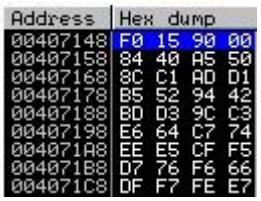
Api correcta: ***GetProcAddress***

No presenta mayor dificultad, asi que la ponemos alli mismo en ImpRec, doble clic sobre ella y elegimos de la lista el nombre de la Api o bien mediante el plugin de ***ASProtect 1.22***, ahora en la ventana log veremos que nos felicita, picamos ***Fix Dump*** y elegimos nuestro dumpeado que yo llamé ***dumped.exe***, nos creará ***dumped_.exe*** y este será el que ahora probaremos, hacemos doble clic sobre él y...



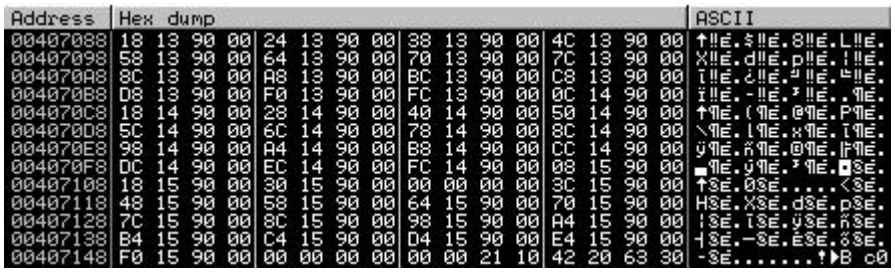
pues empiezan nuestros problemas ;(como ya he dicho esta versión de ASProtect tiene Anti-dumps o almenos suele tenerlos como es el caso, carguemos el programa en OllyDbg y traceemos en busca del error:



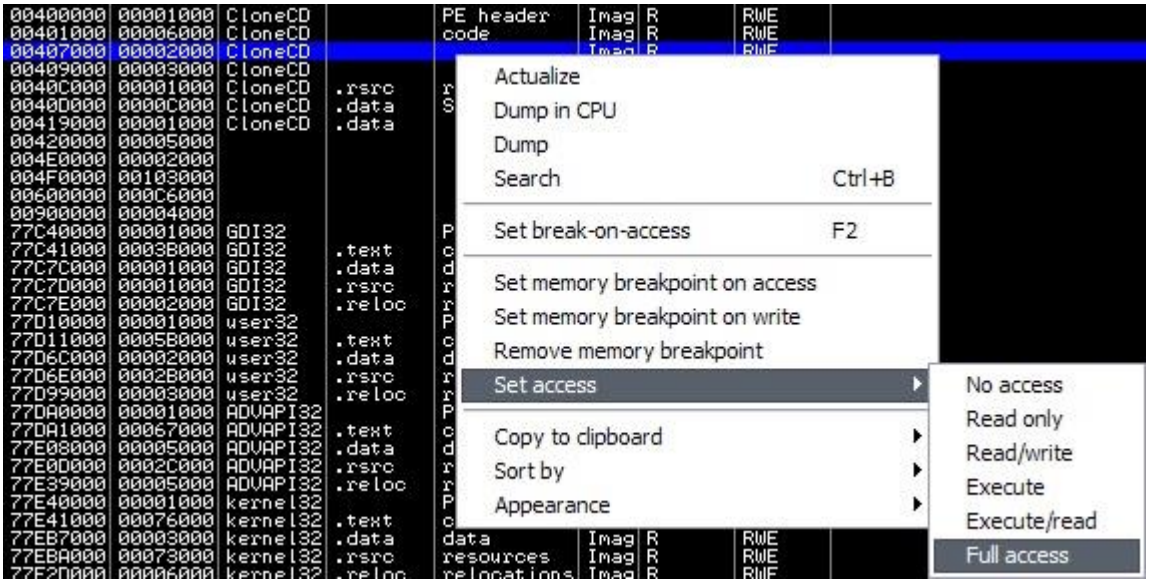


Alli puse una foto del call que nos da problemas acompañada de la ventana dump para ver qué valor esconde la dirección ***[407148]*** que es donde apuntará el call y en este caso es ***:9015F0*** que es una zona de ASProtect que por supuesto en nuestro dumpeado no existe ya que le hemos quitado el packer de el medio, la solución está en ir al original y entrar en dicho call a ver donde nos lleva:

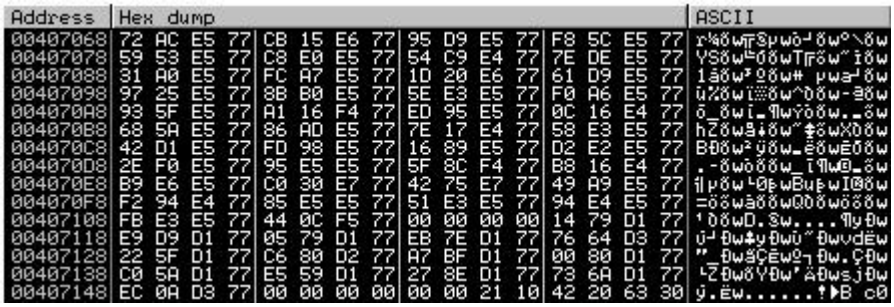
pues a una API, si nos fijamos ademas vemos que el salto va a la IAT¡! claro si el final está justo ahí, miremos la tabla en nuestro dumpeado a ver como esta:



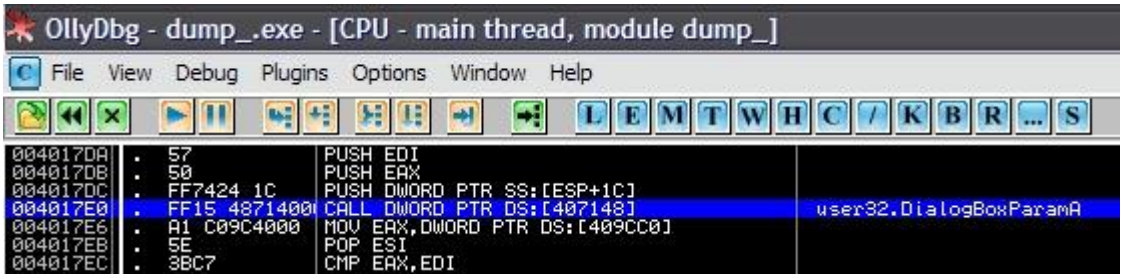
Ahi esta el problema, el ImpRec no nos puso la IAT en su lugar y la dejo tal cual esta en el original, quiza ASProtect nos habrá dejado sin permisos para escribir allí, hagamos lo siguiente, carguemos el original en Olly, reparemos la tabla nopeando el mágico y todo eso como ya sabemos , lleguemos al OEP, ya aqui iremos a la ventana de mamoria Alt+M y seleccionaremos la sección donde irá la IAT antes de dumpear y que estara sin nombre, sabemos que es esta por las direcciones ya que empieza en :47000 que es el inicio de la IAT y con el botón derechi pinchamos en Set Acces -> Full Acces para darle total permiso al ImpRec de escribir allí la tabla a ver si con esto solventamos el tema:



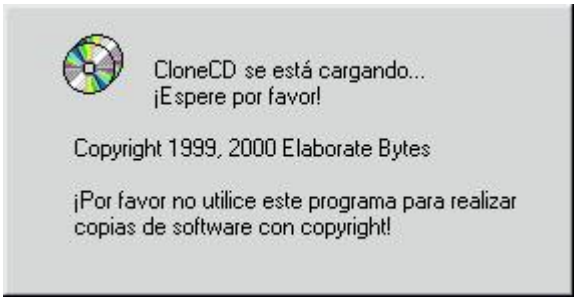
Hecho esto dumpeamos nuevamente, vamos al ImpRec donde reparamos el puntero que quedo inválido, luego hacemos Fix Dump y elegimos el dumpeado, lo abrimos con otro Olly y vamos a ver la iat como está:



Bueno, pues ya esta superado este truquito, ahora si ejecutemoslo de nuevo a ver, vamos traceando como siempre con Olly de momento con F8 para no entrar en los calls, llegaremos al call:

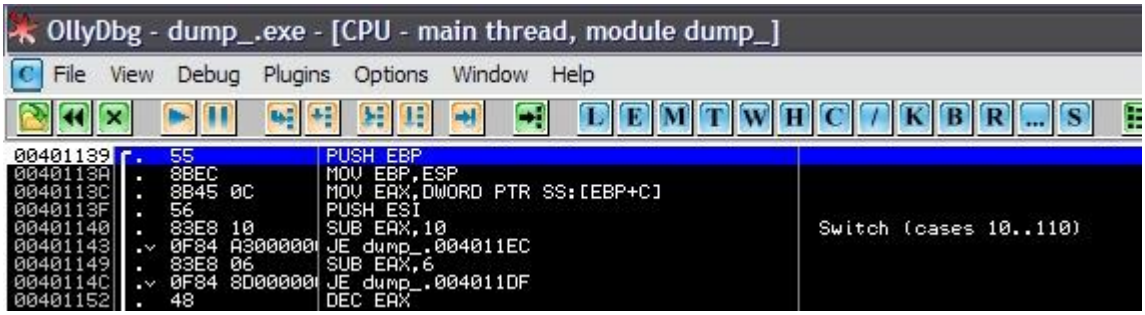


Vemos que ahora si apunta a un api, en concreto a ***DialogBoxParamA*** de la librería ***user32.dll***, si pasamos por el call con ***F8*** aparecerá en pantalla el siguiente baner:



Pero el programa queda como suspendido sin hacer absolutamente nada, parece como si se hubiera quedado colgado o algo parecido, ***Olly*** quedó ***Running*** además ¡qué raro! lo mismo ocurre si ejecutamos el programa fuera de Olly, o sea desde el explorer sin depurarlo, la misma reación, sabemos que los calls a apis del tipo DialogBoxParamA, CreateWindowExA, etc., lo que hacen es ejecutar código del programa original también, o

sea que la llamada a la api antes de retornar entra a ejecutar código como digo del programa, imaginamos que estará pasando algo así lo que haremos seá poner un BPM on Acces en la primera sección cuando lleguemos a la api a ver cuando salta a ejecutar código Olly nos de el controly y podamos tracear en busca del fallo, efectivamente esto ocurre y Olly rompe aqui:



Ahora podemos tracear en busca del error, llegamos a un ret que nos devuelve a la librería ***user32*** pero como tenemos el ***BPM*** damos ***RUN*** y nuevamente parará, así hasta llegar al lugar donde se produzca el error, vemos que estamos en un loop donde se esta cargando el baner y mostrando por pantalla, cuando termina salta a una dirección diferente deshaciendo el loop en el que estaba, caemos aqui:

```
:00401264   55                  PUSH EBP
:00401265   8BEC                MOV EBP,ESP
:00401267   81EC 80010000       SUB ESP,180
:0040126D   53                  PUSH EBX
:0040126E   56                  PUSH ESI
:0040126F   57                  PUSH EDI
:00401270   6A 04               PUSH 4
:00401272   6A FF               PUSH -1
:00401274   FF15 2C704000       CALL DWORD PTR DS:[40702C]          ;Call GetProcAddress
:0040127A   85C0                TEST EAX,EAX
:0040127C   A3 CC9D4000         MOV DWORD PTR DS:[409DCC],EAX       ;Se guarda el valor de retorno
:00401281   0F84 7E010000       JE CloneCD.00401405
:00401287   68 DF5F053A         PUSH 3A055FDF
:0040128C   68 30904000         PUSH CloneCD.00409030               ; ASCII "Once in a while, there is a copy.."
:00401291   8D45 80             LEA EAX,DWORD PTR SS:[EBP-80]
:00401294   68 14914000         PUSH CloneCD.00409114               ; ASCII "%s %8.8x!"
:00401299   50                  PUSH EAX
0040129A   E8 2C090000          CALL CloneCD.00401BCB
:0040129F   8D45 80             LEA EAX,DWORD PTR SS:[EBP-80]
:004012A2   50                  PUSH EAX
:004012A3   E8 A8080000         CALL CloneCD.00401B50
:004012A8   83C4 14             ADD ESP,14
:004012AB   50                  PUSH EAX
:004012AC   8D45 80             LEA EAX,DWORD PTR SS:[EBP-80]
:004012AF   50                  PUSH EAX
:004012B0   FF15 CC9D4000       CALL DWORD PTR DS:[409DCC]          ;Call desencriptador
:004012B6   85C0                TEST EAX,EAX
:004012B8   0F84 42010000       JE CloneCD.00401400
:004012BE   E9 3D010000         JMP CloneCD.00401400
```

bueno pues esta es la zona que buscábamos, aqui paramos cuando termina el bucle que saca el baner inicial, como sabemos es cuando va a sacar el programa principal, esta zona es más que interesante, no estoy seguro pero creo que el programa crea un hilo con CreateThread, si miramos en la ventana de memoria allí vemos que se hace referencia a este en dos lugares distintos: ***stack ot thread xxx*** y ***data block of thread xxx***, coincidiendo en ambos casos las xxx que son el identificador del hilo, o sea que se erfieren al mismo, de lo que no estoy seguro es de la utilidad de ese hilo, se que se crea en ***:401199*** y si ponemos un ***BPX*** en la api el programa no arranca por lo que deduzco que tiene una comprobación por si ponemos algún breakopint en ella, pero un ***HE*** si que funciona y para, así descubrí esta paradita, no se si esto sirve para algo pero pensé por qué no comentarlo? y eso mismo he hecho, pero esto no es lo realmente importante es sólo un apunte,lo que sí interesa es la llamada a ***GetProcAddress*** que en el dumpeado retorna ***EAX=0*** y en el original no, pues este valor de retorno se guarda en ***[409DCC]*** y encima luego se hace una llamada del tipo ***call dwrod ptr [409DCC]*** que desencripta parte del código del programa, veamos unas imagenes que lo haran mas sencillo:

```
OllyDbg - CloneCD.exe - [CPU - thread 00000784, module CloneCD]
File View Debug Plugins Options Window Help

004012AB   50              PUSH EAX
004012AC   8D45 80         LEA EAX,DWORD PTR SS:[EBP-80]
004012AF   50              PUSH EAX
004012B0   FF15 CC9D4000   CALL DWORD PTR DS:[409DCC]
004012B6   85C0            TEST EAX,EAX
004012B8 v 0F84 42010000   JE CloneCD.00401400
004012BE v E9 3D010000     JMP CloneCD.00401400
004012C3   B6 BC           MOV DH,0BC
004012C5   46              INC ESI
004012C6   20D0            AND AL,DL
```

Desde aqui se produce la llamada, si hacemos Follow in Dump y miramos en la ventana dump veremos alli el valor de [409DCC] o sea a donde saltara escrito al reves, veamos:

```
Address   Hex dump                                          ASCII
00409DCC  30 C3 3B 00 00 00 00 00 00 00 00 00 00 00 00 00  0├;.............
00409DDC  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00409DEC  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00409DFC  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00409E0C  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00409E1C  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Vemos que apunta a :3BC330, si ahora seleccionamos los bytes y con el boton derecho picamos eligiendo Follow DWORD in Dump veremos los bytes que va a ejecutar:

```
Address   Hex dump                                          ASCII
003BC330  55 8B EC 53 56 8B 75 0C 8B 5D 08 56 8B CB 8B 15  UïïSVïu.ï]◘Vïïïï§
003BC340  C8 25 3C 00 A1 C4 25 3C 00 E8 A6 FD FF FF 56 53  ╚%<.í─%<.Φª²  VS
003BC350  E8 07 00 00 00 5E 5B 5D C2 08 00 90 55 8B EC 53  Φ•...^[]┬◘.ÉUïïS
003BC360  56 57 BF 54 25 3C 00 8B 1D C4 25 3C 00 A1 C8 25  VW┐T%<.ï#─%<.í╚%
003BC370  3C 00 B9 0C 00 00 00 99 F7 F9 8B F0 85 F6 7E 47  <.╣....Ö÷¨ï-à÷~G
003BC380  8B 07 8B 00 03 03 40 C7 00 01 00 00 00 8B 07 8B  ï•ï.♥♥@%.0...ï•ï
003BC390  00 03 43 04 8B 53 08 E8 84 FF FF FF 8B 45 0C 50  .♥C♦ïS◘Φä    ïE.P
003BC3A0  8B 07 8B 00 03 43 04 8B 4D 08 8B 53 08 E8 42 FD  ï•ï.♥C♦ïM◘ïS◘ΦB²
```

Alli se ven los bytes iniciales ***558B*** que son del tipico ***"PUSH EBP"***, ***"MOV EBP; ESP"*** con los que comienzan la mayoria de las rutinas, o sea que va a ejecutar bastante codigo, lo mas inprtante es saber que esta rutina se encarga de desencriptar parte del codigo del programa principal y si no estoy aquivocado lo hace a partir de ***:4012BE***, veamos con unaa comparacion del antes y el despues de esta zona:

```
OllyDbg - CloneCD.exe - [CPU - thread 0000043C, module CloneCD]
File View Debug Plugins Options Window Help

004012B0   FF15 CC9D4000   CALL DWORD PTR DS:[409DCC]
004012B6   85C0            TEST EAX,EAX
004012B8 v 0F84 42010000   JE CloneCD.00401400
004012BE v E9 3D010000     JMP CloneCD.00401400
004012C3   B6 BC           MOV DH,0BC
004012C5   46              INC ESI
004012C6   20D0            AND AL,DL
004012C8   FD              STD
004012C9   E6 3E           OUT 3E,AL                          I/O command
004012CB   A9 5FF593E8     TEST EAX,E893F55F
004012D0   43              INC EBX
004012D1   D9              ???                                Unknown command
004012D2   48              DEC EAX
004012D3   E8 E3FE9769     CALL 69D811BB
004012D8   F6              ???                                Unknown command
004012D9   4C              DEC ESP
004012DA ^ E0 97           LOOPDNE SHORT CloneCD.00401273
004012DC   02ED            ADD CH,CH
004012DE   6918 6972357D   IMUL EBX,DWORD PTR DS:[EAX],7D357269
004012E4   804B 8B 8C      OR BYTE PTR DS:[EBX-75],8C
004012E8   D339            SAR DWORD PTR DS:[ECX],CL
004012EA   F2:             PREFIX REPNE:                      Modification of segment registe
004012EB   8ECA            MOV CS,DX                          Segment CS is not allowed
004012ED   22A5 7A36DB1A   AND AH,BYTE PTR SS:[EBP+1ADB367A]
004012F3   1AAE BADE6FA1   SBB CH,BYTE PTR DS:[ESI+A16FDEBA]
004012F9   0A4E A3         OR CL,BYTE PTR DS:[ESI-5D]
004012FC   B0 D3           MOV AL,0D3
004012FE   2C 04           SUB AL,4
00401300   9B              WAIT
00401301   29D5            SUB EBP,EDX
00401303   46              INC ESI
00401304   8CC0            MOV AX,ES
00401306   E8 34E52007     CALL 0760F83F
0040130B   49              DEC ECX
0040130C   55              PUSH EBP
0040130D   3F              AAS
0040130E   7A 31           JPE SHORT CloneCD.00401341
00401310   836E 17 6E      SUB DWORD PTR DS:[ESI+17],6E
00401314   AC              LODS BYTE PTR DS:[ESI]
00401315   37              AAA
00401316   BB A73C47D8     MOV EBX,D8473CA7
0040131B   C599 E49F3B16   LDS EBX,FWORD PTR DS:[ECX+163B9FE4]  Modification of segment registe
00401321   0D 5F8AFA16     OR EAX,16FA8A5F
00401326   B8 44E39DD5     MOV EAX,D59DE344
0040132B   43              INC EBX
```

Este es el antes de la zona, si pasamos por encima del call con F8 veremos esto:

```
OllyDbg - CloneCD.exe - [CPU - thread 0000043C, module CloneCD]
File View Debug Plugins Options Window Help

004012B0   FF15 CC9D4000   CALL DWORD PTR DS:[409DCC]
004012B6   85C0            TEST EAX,EAX                          CloneCD.00401401
004012B8   0F84 42010000   JE CloneCD.00401400
004012BE   E9 01000000     JMP CloneCD.004012C4
004012C3   A4              MOVS BYTE PTR ES:[EDI],BYTE PTR DS:[ESI
004012C4   BE D49D4000     MOV ESI,CloneCD.00409DD4
004012C9   68 00010000     PUSH 100
004012CE   56              PUSH ESI
004012CF   6A 00           PUSH 0
004012D1   FF15 28704000   CALL DWORD PTR DS:[407028]
004012D7   85C0            TEST EAX,EAX
004012D9   0F84 21010000   JE CloneCD.00401400
004012DF   BF C89C4000     MOV EDI,CloneCD.00409CC8
004012E4   56              PUSH ESI
004012E5   57              PUSH EDI
004012E6   E8 75070000     CALL CloneCD.00401A60
004012EB   6A 00           PUSH 0
004012ED   57              PUSH EDI
004012EE   E8 BD010000     CALL CloneCD.004014B0
004012F3   83C4 10         ADD ESP,10
004012F6   85C0            TEST EAX,EAX
004012F8   0F84 07010000   JE CloneCD.00401405
004012FE   56              PUSH ESI
004012FF   E8 4C080000     CALL CloneCD.00401B50
00401304   59              POP ECX
00401305   B3 5C           MOV BL,5C
00401307   3898 D49D4000   CMP BYTE PTR DS:[EAX+409DD4],BL
0040130D   74 03           JE SHORT CloneCD.00401312
0040130F   48              DEC EAX
00401310   EB F5           JMP SHORT CloneCD.00401307
00401312   8DB8 D59D4000   LEA EDI,DWORD PTR DS:[EAX+409DD5]
00401318   68 08914000     PUSH CloneCD.00409108                 ASCII "CloneCD.exe"
0040131D   57              PUSH EDI
0040131E   E8 3D570000     CALL CloneCD.00406A60
00401323   59              POP ECX
00401324   85C0            TEST EAX,EAX
00401326   59              POP ECX
00401327   74 34           JE SHORT CloneCD.0040135D
00401329   57              PUSH EDI
0040132A   8D85 80FEFFFF   LEA EAX,DWORD PTR SS:[EBP-180]
00401330   68 D4904000     PUSH CloneCD.004090D4                 ASCII "Please be so kind not tc
00401335   50              PUSH EAX
00401336   E8 90080000     CALL CloneCD.00401BCB
0040133B   83C4 0C         ADD ESP,0C
0040133E   8D85 80FEFFFF   LEA EAX,DWORD PTR SS:[EBP-180]
```

Ahora si que se parece mas a un codigo, antes era totalmente ilegible y ahora todo ha tomado cuerpo, se ven las instrucciones correctamente y no como antes, la disyuntiva surge ahora ¿como evitamos este truco? o bien podemos añadir un injerto que cree la seccion en memoria mediante ***VirtualAlloc***, bien dejando los bytes en un archivo ****.bin*** o bien en una nueva seccion que añadiriamos con ***Topo***, luego como segunda opcion seria dejar que se desencripte y luego dumpear, esto no se si funcionara bien ya que no estamos en el ***OEP***, y la pila puede quedar descuadrada, no se probaremos primero la mas sencilla, o sea, la segunda opcion: dumpear despues de que la seccion este desencriptada, podemos poner un ***BPM on Write*** en toda la seccion para saber si vuelve a desencriptar mas codigo despues o bien con este call ya quedo todo resuelto, si ponemos el ***BPMW*** vemos que ya no escribe mas, asi que haremos una cosa, dumpeamos desde ***:4012B6*** que es la siguiente instruccion al call desencriptador, le reparamos la ***IAT*** con ***ImpRec***, como me da problemas ya que no la salva bien, hare una cosa: como tengo el otro dumpeado con la tabla correcta ya abro dos Olly's en uno cargo el dumpeado con la tabla reparada y en el otro el que acabo de dumpear y que no la tiene, en el primero de ellos (con tabla reparada) voy a la direccion ***:407000*** y selecciono toda la tabla hasta ***:407148*** y hago un ***Binary Copy***, luego voy a la misma direccion del otro y hago un ***Binary Paste***, luego salvo el archivo y ya esta soventado este tema, ahora cargamos el archivo dumpeado con la tabla reparada y desencriptado en Olly (hemos cerrado ya los otros dos Olly's), y comienzo el traceo a ver que me encuentro, de entrada ya veo que despues del ***call GetProcAddres*** hay que nopear el salto para que no nos tire fuera, cambiaremos:

```
:00401281   0F84 7E010000     JE dump_.00401405     POR:
:00401281   909090909090      NOP's
```

Seguimos traceando, llegamos al call desencriptador que logicamente no se debe ejecutar, lo nopeamos tambien, como justo despues hay un salto malo que nos tira fuera tambien lo nopeamos, este otro cambio sera:

```
:004012B0   FF15 CC9D4000   CALL DWORD PTR DS:[409DCC]
:004012B6   85C0            TEST EAX,EAX
:004012B8   0F84 42010000   JE dump_.00401400
```

POR:

```
:004012B0   909090909090     NOP's
:004012B6   85C0             TEST EAX,EAX
:004012B8   909090909090     NOP's
```

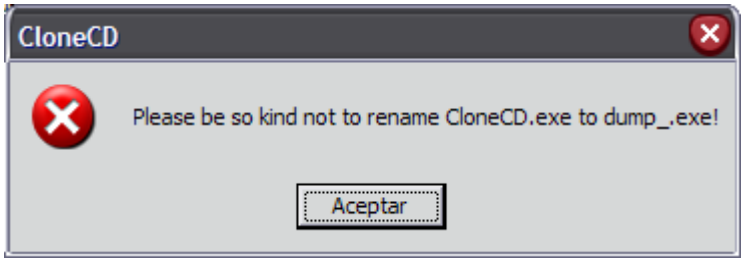
Y evitamos el siguiente error, pero el progama todavia no arranca, siguiendole la pista en el ejecutable original empaquetado llegamos a la conclucion de que el programa parincipal sale en ***:4013D4 -> :4010C3*** (subcall) si seguimos traceando en el dumpeado llegamos a esta parte:

```
:004012EE   E8 BD010000      CALL prueba.004014B0  ;Realiza alguna comprobacion
:004012F3   83C4 10          ADD ESP,10
:004012F6   85C0             TEST EAX,EAX
:004012F8   0F84 07010000    JE prueba.00401405    ;Si no le gusta nos tira fuera..!!
```

Pues ahí tenemos otro problema, este call mira algo que no he seguido porque nopeé directamente el salto:

```
:004012F8   0F84 07010000    JE prueba.00401405    POR:
:004012F8   9090 90909090    NOP's
```

Y continuamos, ahora vemos este mensaje:



Está claro que ahora no le gusta el nombre, esto no tiene problema seguimos traceando y al ommento llegamos a la comprobación, parece que las tiene todas juntas y es precisamente en la zona recién desencriptada por el call anterior, fijáos donde llegamos:

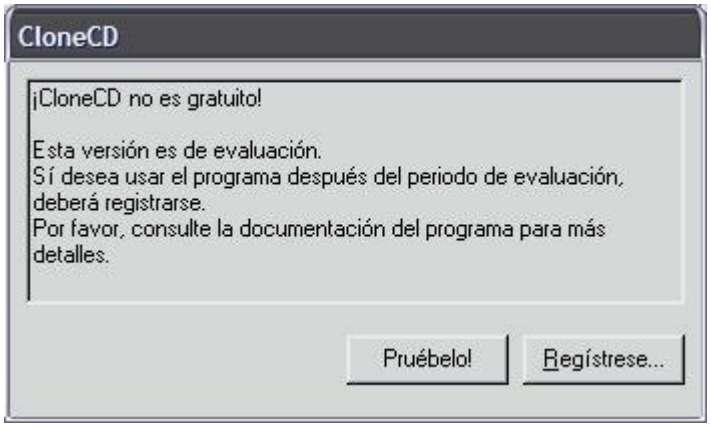
```
:00401312   8DB8 D59D4000    LEA EDI,DWORD PTR DS:[EAX+409DD5]
:00401318   68 08914000      PUSH dump_.00409108          ;  ASCII "CloneCD.exe"
:0040131D   57               PUSH EDI
:0040131E   E8 3D570000      CALL dump_.00406A60          ;Aqui comprueba el nombre del archivo...!!!
:00401323   59               POP ECX
:00401324   85C0             TEST EAX,EAX
:00401326   59               POP ECX
:00401327   74 34            JE SHORT dump_.0040135D
:00401329   57               PUSH EDI
:0040132A   8D85 80FEFFFF    LEA EAX,DWORD PTR SS:[EBP-180]
:00401330   68 D4904000      PUSH dump_.004090D4     ; "Please be so kind not to rename CloneCD.exe to %s!"
```

Vemos un push de la cadena CloneCD.exe y un ***PUSH EDI*** cuto valor carga dos líneas antes en ***:401312***, si traceamos hasta llagar a esta istrucción y hacemos un ***Follow in Dump*** allí vemos ***dump_.exe*** que es el nombre del archivo, o sea que se le pasan estos argumentos al ***CALL 406A60*** que compara ambas cadenas, si son distintas retorna ***EAX=1*** y vemos el mensaje, esto podemos solucionarlo fácilmente cambiando:

```
:00401327   74 34            JE SHORT dump_.0040135D    POR:
:00401327   EB 34            JMP SHORT dump_.0040135D
```

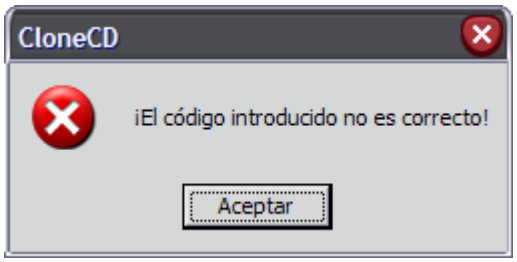
Y ya no vemos más el mensaje, si lo ejecutamos ahora el programa cargaaaaaaaaaaa...!!!

Bueno pues ya hemos derrotado al anti-dump, no podemos decir que ha sido ni fácil ni difícil, aomenos para mis habilidades, podría haber sido peor pero también mejor, un término medio, si como digo ejecutamos el progama ahora sí que veremos la ***nag*** típica de los ***CloneCD***, jeje..

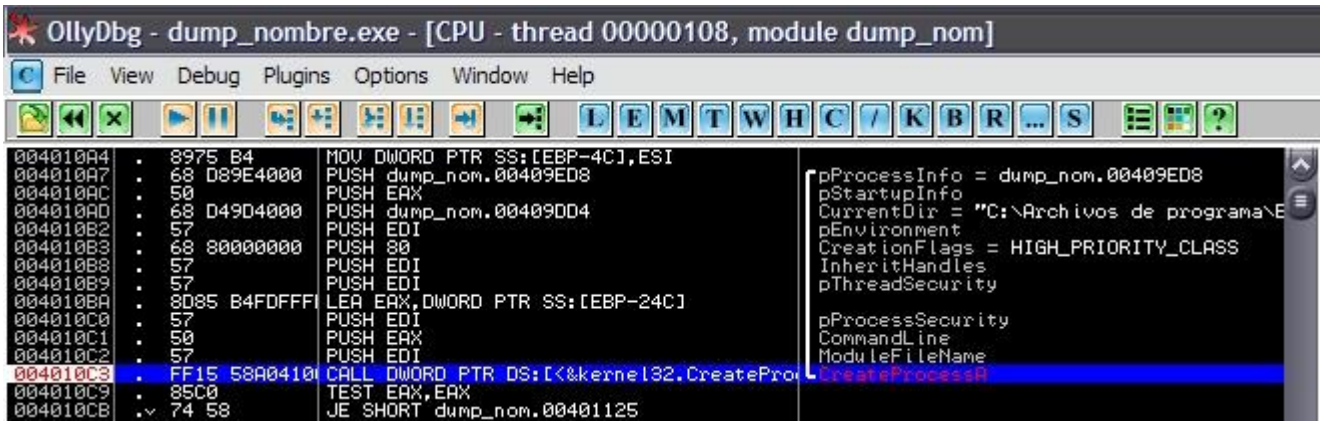


Ahora vamos a ***crackear*** el archivo ya que estamos ;DDD

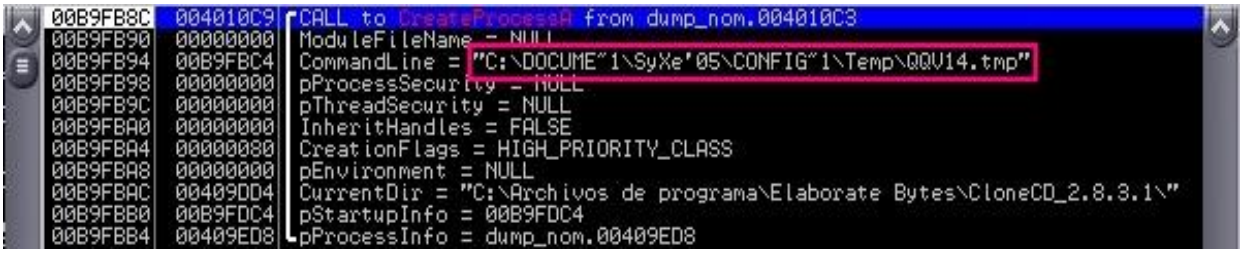
Aquí las estrategias pueden ser varias ya que nos da la posibilidad de introducir un serial podemos atacar por aqui, podemos bien eliminar la nag y las limitaciones, en fin hay varios ataques, de momento lo intentaremos por el serial, cargamos el programa en ***Olly***, ponemos en el ***CommandBar*** un ***BP MessageBoxA*** y le damos a ***F9*** para que corra, introducimos un serial cualquiera con nuestro nombre y aceptamos, aparece el mensaje de error:



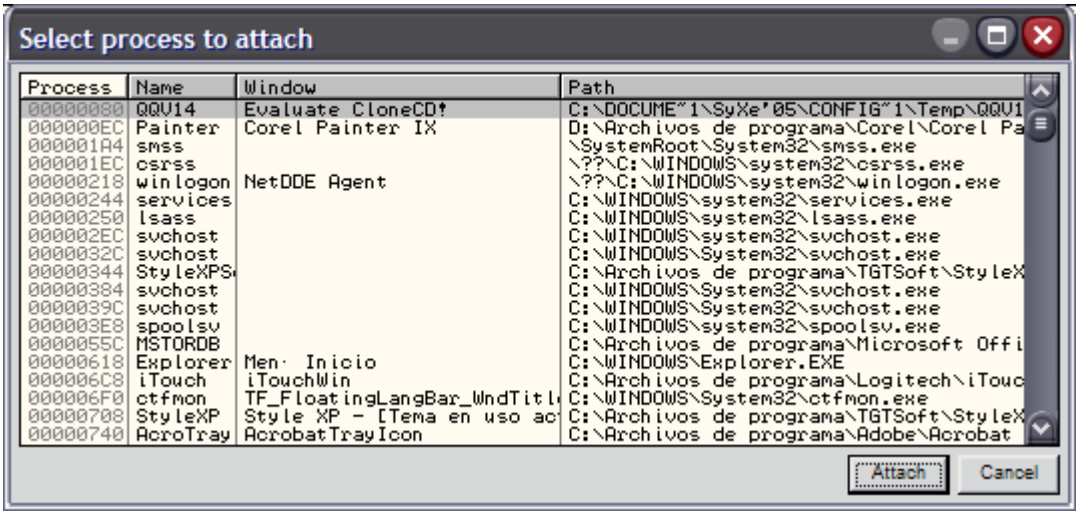
Pero Olly no para ¿? Estoy empezando a pensar que todo se cuece en un archvo temporal ya que si cerramos el Olly el programa queda en pantalla lo que indica que no lo estamos depurando, luego es un temporal, veamos si esto es cierto, miremos lo que nos encontramos:



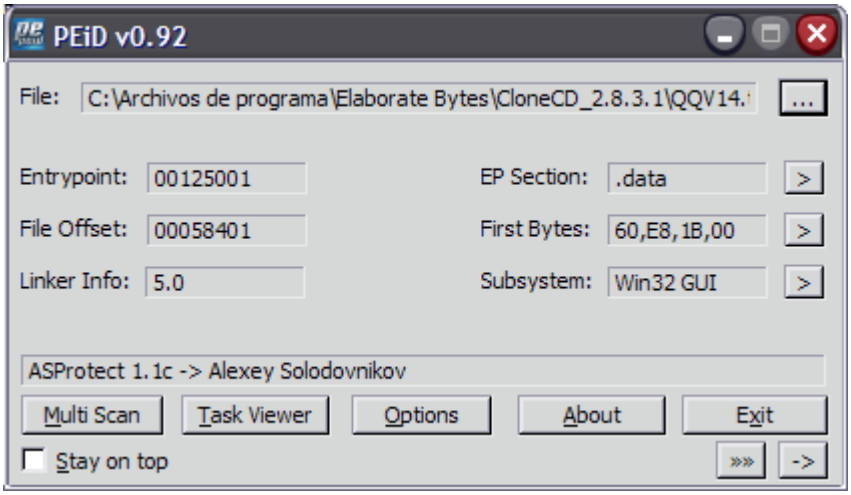
Ahi esta nuestra llamada, crea un segundo proceso mediante una llamada a CreateProcessA, si entramos en la primera linea podemos ver esto en es stack:



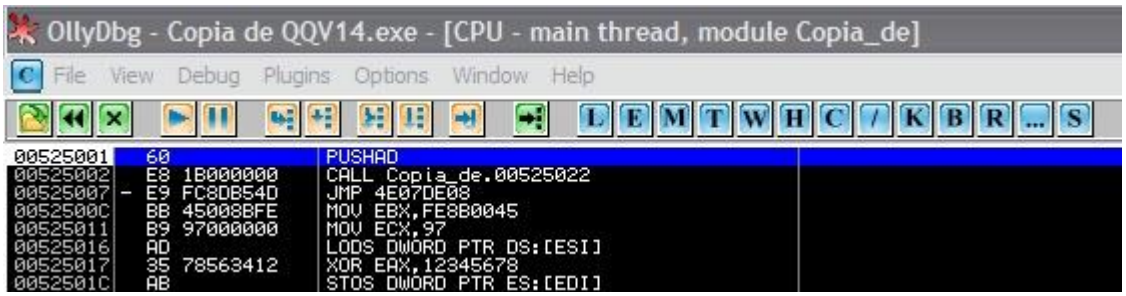
Vemos los parametros con los que se llamo la api, entre ellos la ruta y el nombre del archivo a crear, es este caso sera "C:\Documents and Settings\SyXe'05\Configuración local\Temp //QQV14.tmp" si vamos alli vemos que efectivamente el archivo existe y es la verdadera aplicacion, ahora veremos como lo atacamos, ademas si vamos a Olly a ***File -> Attach*** mirad que vemos:



Anda que el nombre lo han escondido o algo ***Evaluate CloneCD!*** parece mentira que sean tan descarados, o sea que el CloneCD que hemos desempaquetado es una lanzadera de la verdadera aplicacion que este archivo, lo atachearemos de momento a ver que podemos hacer, de momento mirad esto:



También está empaquetado con ASProtect, jodeeeeeeeer..!! y con la misma version, ahora la pregunta es ¿como lo desempaquetamos? madre mia en que follon me estoy metiendo porque esto yo nolo sabia, nisiquiera sabia que tenia un temporal que era la aplicacion principal, si le cambiamos el nombre a QQV14.exe aparece el icono del CloneCD pero como con gota de agua igual que los archivos ocultos, si intentamos ejecutarlo no se abre y nodice nada, abramoslo don Olly:



Es el tipico EP de ASProtect, intentaremos desempaquetarlo a ver que pasa, siguiendo exactamente el mismo metodo descrito antes, llegaremos a la ultima excepcion que anotaremos:

Ultima excepcion: D00509
OEP: 401000
IAT => Inicio: 4F4124
Final: 4F4F24
Largo: E00

Vemos que esta tabla si tiene referencias a la libreria elbycdio.dll, jeje.. estamos en lo cierto, acordaos antes cuando os dije que me extrañaba no encontrarla entre las importaciones de la iat del otro exe, claro! si es que era una simple lanzadera de la verdadera aplicacion ¡nos querian engañar! pero esta vez no han podido,

Primer puntero redirigido de la iat [4F4380]=D10334, perdonad que vaya tan rapido pero el metodo es totalmente igual al anterior, sino no acabaremos nunca este tut, Reiniciemos con Ctrl+F2 pongamos un BPMW en esta direccion, vamos a buscar el magico:

Call semi-magico: CFF0D6

Reiniciamos y nopeamos para reparar la iat, dumpeamos y luego vamos con ImpRec a reparar la tabla, vaya si nopeo el magico entramos en una excepcion de la que no se puede recuperar Olly, nos haremos un script que mire si hemos parado en el call magico y de ser asi que cambien el EIP para que apunte a la siguiente instruccion

```
---------------------------/ /---------------------------/
start:
eob break
run

break:
cmp eip, 0cff0d6
jne final
mov eip, eip, 0cff0db
jmp start

exit:
ret
---------------------------/ /---------------------------/
```

Bueno nos sale un mensaje de error pero nos vale para recuperar el ***95%*** de la tabla, jeje.. lo que haremos sera copiarla a un lugar seguro como iat.txt o algo asi para luego llegar normalmente al OEP y pegarla con un Binary Paste, vamos al ImpRec y nos dice que quedaron concretamente, pero que digo! no era un error era una excepcion, no hemos sido detectados, jeje.. superamos la excepcion y asi hasta la ultima antes del ***OEP***, ponemos el ***BPM*** en la primera seccion y con ***Shift+F9*** superamos esta ultima, lelgamos al OEP y dumpeamos ya, cuidado no nos olvidemos de activar las excepciones para parar en ellas sino no llegaremos al OEP. Bien tenemos ***12*** entradas por resolver, algunas incluso nos la arregla el propio Import mediante un ***Tracer level1***, otras como estas no:

Entrada 1:

Iat rva: F4430
Apunta a: CFC208

```
DISASSEMBLER / HEX VIEWER
00CFC208    push ebp
00CFC209    mov ebp,esp
00CFC20B    mov edx,[ebp+C]
00CFC20E    mov eax,[ebp+8]
00CFC211    cmp eax,[D010BC]     // DWORD value: FFFFFFFF
00CFC217    jnz short 00CFC222
00CFC219    mov eax,[edx*4+D011EC]     // DWORD value: FFFFFFFF
00CFC220    jmp short 00CFC229
00CFC222    push edx
00CFC223    push eax
00CFC224    call 00CF43EC     // = kernel32.dll/0191/GetProcAddress
00CFC229    pop ebp
00CFC22A    retn 8
```

Api correcta: ***GetProcAddress***

La verdad que quitando esta y otra que era ***CloseHandle*** que me arreglo el ***Tracer Level1*** quedan ***10*** de lo mas jodidas ya que no las redirige de la manera tradicional, pues para en otra zona donde no se ve ingun salto ni call magico, o sea que esta el tema jodido, podria tracearlas, son solo 10 pero imagino que debe haber un modo mas simple, este otro punto donde redirecciona estas apis es este:

:00CFF182 8902 MOV DWORD PTR DS:[EDX],EAX

Puedo probar a investigar un poco la rutina de ASProtect, veo que modificando un salto no consigo mucho porque no logro descubrir el nombre de la api, pero si veo la ***dll*** a la que pertenece la funcion tratada, como hay tres grupos de apis sin resolver puedo mirar almenos a que libreria pertenecen:

:4F458C - 4F4594 => Punteros de ***version.dll***
:4F4670 - 4F4674 => Punteros de ***comdlg32.dll***
:4F4914 - 4F4924 => Punteros de ***shell32.dll***

Si traceo un poco mas llego a la zona donde desencripta el nombre de cada api que esta procesando en ese momento y que sera la siguiente en poner el la tabla o bien redirigirla despues

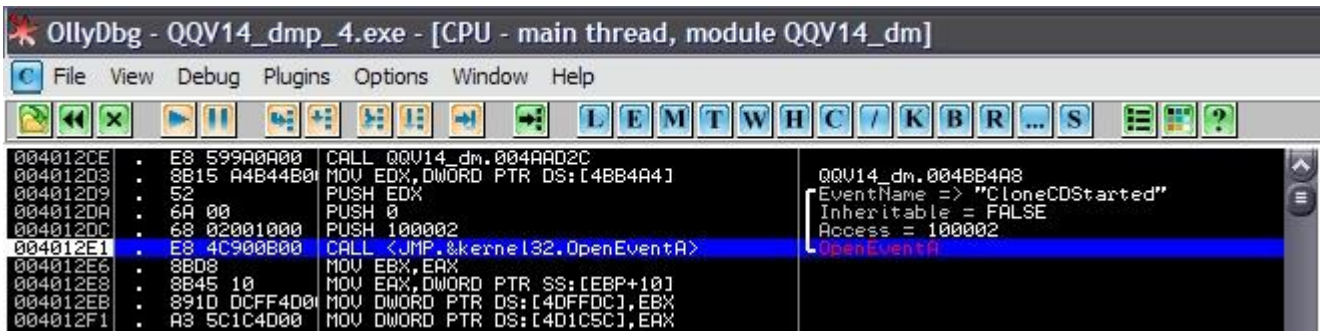
```
00CF8BBD  49      DEC ECX
00CF8BBE  7C 0B   JL SHORT 00CF8BCB
00CF8BC0  8B048E  MOV EAX,DWORD PTR DS:[ESI+ECX*4]
00CF8BC3  33048A  XOR EAX,DWORD PTR DS:[EDX+ECX*4]
00CF8BC6  89048F  MOV DWORD PTR DS:[EDI+ECX*4],EAX
00CF8BC9  EB F2   JMP SHORT 00CF8BBD
```

Este loop esta en el interior de varios calls de la propia rutina, esta es la ruta

:00CFC160 E8 ABCEFFFF CALL 00CF9010 ; pone la API en la pila
:00CF914D E8 2EFAFFFF CALL 00CF8B80 ; te da el nombre de la API
Loop anterior => desencripta el nombre de las apis

Este es un mini esquema de la rutina que rellena la ***IAT***, pero como digo lo más que conseguí fue ver el nombre de la ***dll*** que está tratando pero en ningún momento puede ver que ***api*** concreta era con lo que no pude injertarle para reparar la tabla, esto me limitaba bastante así que pensé: ¿porqué me voy a empecinar por este camino cuando en realidad hay cientos para hacer lo mismo? y me puse en marcha ;DDD

Lo primero que hice fue primero de todo hacerle creer al programa que ha sido lanzado desde CloneCD.exe, recordad que trabajamos con un archivo temporal que en realidad es la propia aplicacion, lo que ocurre sergun he visto es que realiza un chequeo para ver si ha sido lanzado por la lanzadera que es como digo ***CloneCD.exe***, esto no lo digo por decir, ni tampoco me lo invento, lo deduzco de esta llamada:



De aqui deduci que deberia mirar en algun momento si ha sido lanzado, quiza ese ***CloneCDStarted*** le indique esto pero como digo es una mera suposicion, lo que si esta claro es que la llamada a ***OpenEvent*** debe retornar ***EAX=!0*** porque sino nos tira fuera, para evitar esto lo que haremos sera jumpear este salto:

:004012F8 7511 JNZ SHORT Copia_de.0040130B

y ahora el programa empezara a cargar, es momento de recuperar la ***IAT***, jeje. el metodo es poco tradicional y quiza un poco chapuza pero funciona..!! el tema esta en poner un ***BPX*** en los ***10*** punteros que no podemos resolver con ***Import Reconstructor***, como son ***only 10*** vamos a por ellos manualmente:

```
OllyDbg - Copia de QQV14.exe - [CPU - main thread, module Copia_de]
File View Debug Plugins Options Window Help

004BA69A - FF25 F0484F00  JMP DWORD PTR DS:[4F48F0]      GDI32.StretchBlt
004BA6A0 - FF25 F4484F00  JMP DWORD PTR DS:[4F48F4]      GDI32.UnrealizeObject
004BA6A6   CC             INT3
004BA6A7   CC             INT3
004BA6A8 - FF25 14494F00  JMP DWORD PTR DS:[4F4914]
004BA6AE - FF25 18494F00  JMP DWORD PTR DS:[4F4918]
004BA6B4 - FF25 1C494F00  JMP DWORD PTR DS:[4F491C]
004BA6BA - FF25 20494F00  JMP DWORD PTR DS:[4F4920]
004BA6C0 - FF25 24494F00  JMP DWORD PTR DS:[4F4924]
004BA6C6   CC             INT3
004BA6C7   CC             INT3
004BA6C8 - FF25 F84B4F00  JMP DWORD PTR DS:[4F4BF8]      USER32.ActivateKeyboardLayout
004BA6CE - FF25 FC4B4F00  JMP DWORD PTR DS:[4F4BFC]      USER32.AdjustWindowRectEx
```

Hacemos esto con los otros ***5 punteros*** que quedaron sin resolver, ahora lo dejamos correr con ***F9***, no olvidar jumpear el salto en ***:4012F8*** sinó no cargará el programa, ahora irán saltando los ***BPX*** que tenemos puestos, cuando paremos en uno lo quitamos con ***F2*** y picamos a ***F7*** para seguir el puntero a ver donde nos lleva, caemos en una rutina de ASProtect que tiene la propiedad de retornar justo a la primera instrucción de la api, lo que quiere decir que si nos ponemos en el ret y miramos al ***stack*** veremos esta sustancial ***info***:

```
0012F300  0040B9D1  CALL to GetFileVersionInfoSizeA from Copia_de.0040B9CC
0012F304  0012F318  FileName = "C:\Archivos de programa\Elaborate Bytes\CloneCD_2.8.3.1\
0012F308  0012F420  pHandle = 0012F420
0012F30C  004D243C  ASCII "C:\Archivos de programa\Elaborate Bytes\CloneCD_2.8.3.1\"
0012F310  004D213C  ASCII "C:\DOCUME~1\SyXe~05\CONFIG~1\Temp\IMAGE.CCD"
0012F314  004D1FFC  Copia_de.004D1FFC
0012F318  415C3A43
0012F31C  69686372
```

Mierda de foto!! no se lee bien pero en rojo pone ***GetFileVersionInfoSizeA*** que es la api a la que debería apuntar, ademas nos dice hasta de donde se hizo la llamada que no corresponde con la direccion del ***jmp dword ptr [xxxxx]*** ya que el puntero está dentro de un call que es el que le da la dirección de retorno, es por esto, de hecho si vais a la dirección ***:40B9CC*** y hacemos ***Follow*** veremos allí nuestro puntero ;DDD

Ahora que tenemos el nombre de la api procedamos a reparar la entrada, sabemos que el punero era este:

:004BA422 - FF25 90454F00 JMP DWORD PTR DS:[4F4590]

nos situamos sobre el y hacemos un ***Follow in Dump*** para caer en la iat justo en esta entrada, ahora en el ***CommandBar*** pnemos el nombre de la api con un interrogante delante, picamos ***Enter*** y nos da su direccion, asi:

```
Command ?GetFileVersionInfoSizeA   HEX: 77BD15E3 - DEC: 2008880611 - ASCII: w½□ã
```

esta direccion la ponemos el la iat escrita al reves:

```
Address   Hex dump                                          ASCII
004F4590  E3 15 BD 77 88 41 D1 00 00 00 00 00 00 00 00 00   ã§wêAÑ.........
004F45A0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
004F45B0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
004F45C0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
004F45D0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
```

y ya nos queda una menos, alli en rojo esta la direccion de la api escrita al reves, pero mi ordenata es miope o algo porque no ve bien las fotos y se ve borroso el color rojo ;(no se si a ti te pasara ¿?

Bueno pues para no tener que hacer esto 9 veces mas ya que es tonteria, pues haciendo una sabremos hacer el resto pasaremos al ataque, cuando tengamos todas reparadas vamos al ***ImpRec***, seleccionamos el proceso del programa, le ponemos los datos correctos:

Pero antes aprovechamos para dumpear cuando estemos en el ***OEP*** y le llamamos ***temp.exe*** para diferenciarlo bien, y depues si vamos al ***Import Reconstructor*** a darle los datos siguientes:

Picamos en ***Get Imports*** y nos saldran dos punteros invalidos, el de ***GetProcAddress*** que reparamos con el plugin y el de ***CloseHandle*** que repararemos con el ***Tracer Level 1***, luego picamos ***Fix Dump***, elegimos el dumpeado a reparar que sera ***temp_.exe*** que es el que deberemos probar... Hacemos doble clic y...

Allí está nuestro programa rulandoooooo...!!!

Pero no nos apresuremos porque según que toquemos nos Reinicia el ordenador!!! hijo de p.... qué pasará? algún puntero de la iat estará jodido? para quitarnos la duda copiaremos la iat del dumpeado en el proceso original (del archivo temporal claro y no de la lanzadera esto se entiende) y probamos las funciones, si chuta sabemos que esto no es y si no pues ya tenemos por donde empezar.

Pues bien acabo de copiar la iat del dumpeado al original, he utilizado una función y parece que todo rula bien, pero toco otra y bouuuuumm...!!! me reinicia el ordenador quejándose de nosequé dirección, ahora ya sabemos que no tenemos una IAT correcta, quiza sea que necesite el código de la rutina de ASProtect de donde obtiene la direccion de la api a la que saltara pues son bastante grandes y quizá hagan mas cosas, esto abre otra vez la puerta de par en par a un posible injerto, pero ya veremos porque llevo mucho pero que mucho rato intentando encontrar una solución, de hecho llevamos escritas 19 páginas y aún se resiste a caer, quien lo diría eh! un ASProtect 1.1c que parecía peso pluma (y quizá lo sea pero no para mi)

Por si alguien tiene dudas como hacer que salten ciertor punteros os pongo como lo hice yo:

:004BA41C - FF25 8C454F00 JMP DWORD PTR DS:[4F458C]
API: version.**GetFileVersionInfoA** (al iniciar el programa)
:004BA422 - FF25 90454F00 JMP DWORD PTR DS:[4F4590]
API: version.**GetFileVersionInfoSizeA** (al iniciar el programa)
:004BA428 - FF25 94454F00 JMP DWORD PTR DS:[4F4594]
API: version.**VerQueryValueA** (al iniciar el programa)
-> en binario: (2D 16 BD 77 E3 15 BD 77 7D 16 BD 77)

:004BA4C0 - FF25 70464F00 JMP DWORD PTR DS:[4F4670]
API: comdlg32.**GetOpenFileNameA** (Archivo -> Abrir Imagen .CDD)
:004BA4C6 - FF25 74464F00 JMP DWORD PTR DS:[4F4674]
API: comdlg32.**GetSaveFileNameA** (Archivo -> Seleccionar Sonido)
-> en binario: (90 23 36 76 CF 90 36 76)

:004BA6A8 - FF25 14494F00 JMP DWORD PTR DS:[4F4914]
API: SHELL32.**SHBrowseForFolderA** (cambiar Skins)
:004BA6AE - FF25 18494F00 JMP DWORD PTR DS:[4F4918]
API: SHELL32.**SHGetDesktopFolder** (cambiar Skins)
:004BA6B4 - FF25 1C494F00 JMP DWORD PTR DS:[4F491C]
API: SHELL32.**SHGetMalloc** (cambiar Skins)
:004BA6BA - FF25 20494F00 JMP DWORD PTR DS:[4F4920]
API: SHELL32.**ShellExecuteA** (Visitar pagina Elaborate Bytes)
:004BA6C0 - FF25 24494F00 JMP DWORD PTR DS:[4F4924]
API: SHELL32.**SHGetPathFromIDListA** (cambiar Skins)
-> en binario: (C2 35 4B 77 1D C5 3C 77 56 B0 3F 77 B9 29 40 77 31 27 40 77)

No creo que nadie tenga problemas pero por si las moscas ahi os he puesto las apis a las que apuntan los punteros que estan redirigidos y al final de cada bloque donde pone ***en binario*** son los bytes asociados a cada grupo de apis completo para ponerlos en la iat y se repararlos de esta otra manera, espero no liaros.

En :534B00 hay espacio para el injerto

busquemos las direcciones de la iat donde se guardan las direcciones de VirtualAlloc, CreateFileA y ReadFile:

VirtualAlloc en [4F4544]
CreateFileA en [4F438C]
ReadFile en [4F44E4]

necesitaremos algunas variables sonde almacenar el nombre del archivo, el tamaño de la sección a crear. Estas serán por ejemplo las siguientes:

en [534BC0] estará el nombre del archivo injerto_temp.bin
en [534BD0] estará la longitud del archivo que son 8000h bytes

Con estos datos podemos construir nuestro injerto que será más o menos este:

```
pushad                  ;Salvamos los registros
push 4                  ;Acceso completo
push 2000               ;Pedimos que reserve la zona
push 8000               ;Tamaño seccion reservada
push 0D10000            ;Direccion donde la queremos
call dword ptr [4F4544] ;Llamada a VirtualAlloc
test eax, eax
push 4                  ;Acceso completo
push 3000               ;Le decimos que nos cree ya la zona
push 8000               ;Tamaño seccion reservada
push 0D10000            ;Direccion donde la queremos
call dword ptr [4F4544] ;Llamada a VirtualAlloc
test eax,eax
push 0                  ;argumentos de la funcion
push 0                  ; . . .
push 3                  ; . . .
push 0                  ; . . .
push 0                  ; . . .
push 80000000           ; . . .
push 534BC0             ; puntero al nombre del archivo
call dword ptr [4F438C] ;Llamada a CreateFileA
test eax, eax
push 0                  ;primer argumento de la funcion
push 534BD0             ;puntero a la longitud del archivo a leer
push 8000               ;tamaño a leer
push 0D10000            ;buffer de almacenamiento
push eax                ;handle del archivo
call dword ptr [4F44E4] ;Llamada a ReadFile
test eax, eax
popad                   ;Restauramos los registros
jmp <Entry_Point>       ;Saltamos al EntryPoint
```

Esto más o menos lo que hará será reservar un espacio de memoria de tamaño ***8000h*** bytes apartir de la dirección ***:0D10000h***, luego le pedirá este espacio de memoria al sistema (parámetro ***MEM_COMMIT***), acto seguido va a obtener el maejador al archivo ***temp.bin*** con ***CreateFileA***, en ***EAX*** obtenemos el ***handle*** con el que luego accedemos al archivo y leemos su contenido con ***ReadFile***, ademas utilizaremos como buffer de salida el espacio de memoria creado, o sea que se copiaran alli los bytes contenidos en el archivo ***temp.bin***, restaurara los registros y saltara al ***Entrypoint***, previamente con ***PEditor*** cambiaremos el ***EP*** actual en ***:401000*** hacia la direccion del injerto ***:534B00***, de esto no adjunto imagen porque no tiene ningun sentido y seria alargar todavia este interminable tuto que voy loco por acabar (si puedo, jeje..).

No perdamos mas tiempo y pongamos alli el injerto, no olvidar copiar el nombre del archivo bin y su tamaño en las direcciones arriba indicadas para que todo funcione.

Pues hecho el injerto y con todo en su sitio lanzo el programa y peta, miro y faltaban los dos punteros de la iat que podemos reparar, CloseHandle y GetProcAddress, nada los pongo a mano y salvo el archivo con otro nombre, ejecuto este otro que tiene ya el injerto y... aparte de dar algún pequeño error también me reinicia el ordenador..!!! esto es para pensar cosas raras "-#%&"D@$#"...

Después de muchos intentos, muchos creedme con su consecuente reinicio de la máquina logré aislar el problema, pues sí, la entrada reparada como GetProcAddress es la que nos está jodiendo, hice la prueba con el original, le cambié solo este puntero y Se Reinició...!!! luego, hemos pillado el error, es un buen comienzo, ahora queda saber porqué se comporta así cuando la entrada debe ir allí claramente, quizá el injerto haya que hacerlo para este único puntero y reparar el resto pues sabemos que apis son, el puntero conflictivo es este:

004BA230 $- FF25 30444F00 JMP DWORD PTR DS:[4F4430]

si miramos en la ventana dump vemos que [4F4430] contiene el valor 08C2CF00 que visto del revés significa 00CFC208, o sea que apuntará a la dirección esta, que es una zona de ASProtect, si miramos allí vemos:

```
:00CFC208  55              PUSH EBP
:00CFC209  8BEC            MOV EBP,ESP
:00CFC20B  8B55 0C         MOV EDX,DWORD PTR SS:[EBP+C]
:00CFC20E  8B45 08         MOV EAX,DWORD PTR SS:[EBP+8]
:00CFC211  3B05 BC10D000   CMP EAX,DWORD PTR DS:[D010BC]
:00CFC217  75 09           JNZ SHORT 00CFC222
:00CFC219  8B0495 EC11D000 MOV EAX,DWORD PTR DS:[EDX*4+D011EC]
:00CFC220  EB 07           JMP SHORT 00CFC229
:00CFC222  52              PUSH EDX
:00CFC223  50              PUSH EAX
:00CFC224  E8 C381FFFF     CALL 00CF43EC          ; JMP to kernel32.GetProcAddress
:00CFC229  5D              POP EBP
:00CFC22A  C2 0800         RETN 8
```

Pues la verdad que no se que tipo de error puede ser porque se ve claramente que va a ***GetProcAddress***, para asgurarme de esto reparo los punteros excepto este que lo dejo asi y el original funciona con esa ***IAT***, es más si copio la iat entera del dumpeado al original excepto el puntero a ***GetProcAddress*** tambien funciona lo que deja al descubierto que nuestra teoria es correcta, no se porque motivo esta entrada es el que nos esta jodiendo, a esto se me ocurre reparar toda la iat excepto este puntero en un nuevo dumpeado y hacer un injerto para que trabaje el alli con la zona de ***ASProtect*** cargada en tiempo de ejecucion a ver si esta es la buena de todas que ya esta bien la bromita!! entonces repetire todos los pasos del script y todo hasta obtener un dupeado que llamaré ***ultimo.exe*** al que le repararé toda la iat con ***ImpRec*** menos el puntero conflictivo para el que haremos un ***injerto*** a ver si asi trabaja bien el muy jodido, la zona que debemos dumpear empieza en ***:CF0000*** y tiene un tamaño de ***18000h*** bytes, realizamos la misma operacion con ***HexWorkShop*** de pegado especial en ***CF_TEXT*** y le damos el nombre ***final.bin***, luego desviamos el ***EP*** y acomodamos el injerto anterior para que ahora cargue este otro, esto no es difícil sabiendo hacer el anterior pues solo debemos modificar algún que otro ***PUSH***, el nombre del archivo y su tamaño, nada más el resto sera totalmente igual.

El injerto en este caso lo pondremos a partir de ***:534C00*** y utilizamos las mismas variables pero con diferente dirección y contenido claro está, luego veremos una imagen del mismo si funciona el invento ;DDD

estas variables serán:

en [534CC0] estará el nombre del archivo ***final.bin***
en [534CD0] estará la longitud del archivo que son ***18000h*** bytes

en binario quedan estos bytes (del injerto completo):

60 6A 04 68 D0 07 00 00 68 00 80 01 00 68 00 00 CF 00 FF 15 44 45 4F 00 85 C0 6A 04 68 00 30 00
00 68 00 80 01 00 68 00 00 CF 00 FF 15 44 45 4F 00 85 C0 6A 00 6A 00 6A 03 6A 00 6A 00 68 00 00
00 80 68 C0 4C 53 00 FF 15 8C 43 4F 00 85 C0 6A 00 68 D0 4B 53 00 68 00 80 01 00 68 00 00 CF 00
50 FF 15 E4 44 4F 00 85 C0 61 E9 91 C3 EC FF

Hecho esto probamos a ejecutar el nuevo ***exe*** y todavia no funciona del todo bien, lo cargamos en ***Olly*** y traceamos un poco hasta encontrar que caemos en una llamada a ***RaiseException*** en esta zona:

```
:00498031   837B 04 00        CMP DWORD PTR DS:[EBX+4],0
:00498035   7D 24             JGE SHORT ultimo_1.0049805B        ;Bonito salto...!!!
:00498037   8975 F4           MOV DWORD PTR SS:[EBP-C],ESI
:0049803A   C645 F8 0B        MOV BYTE PTR SS:[EBP-8],0B
:0049803E   8D45 F4           LEA EAX,DWORD PTR SS:[EBP-C]
:00498041   50                PUSH EAX
:00498042   6A 00             PUSH 0
:00498044   8B0D 78194D00     MOV ECX,DWORD PTR DS:[4D1978]      ;  ultimo_1.00484F7C
:0049804A   B2 01             MOV DL,1
:0049804C   A1 F4404900       MOV EAX,DWORD PTR DS:[4940F4]
:00498051   E8 56B20000       CALL ultimo_1.004A32AC
:00498056   E8 65FE0000       CALL ultimo_1.004A7EC0             ;Llamada a RaiseException...!!!
```

que superamos cambiando:

```
:00498035   7D 24     JGE SHORT ultimo_1.0049805B      POR:
:00498035   EB 24     JMP SHORT ultimo_1.0049805B
```

Atención señores que después de este cambio el programa arranca, probamos las funciones y ***NO SE CIERRA***..!!! nos hemos fulminado al puto ***Anti-dump***, jeje.. reconozco que ha sido durísimo pero al final ha valido la pena ;XDDD como me gusta esto, jeje.. aqui os pongo el injerto final que os dije que pondría si funcionaba:

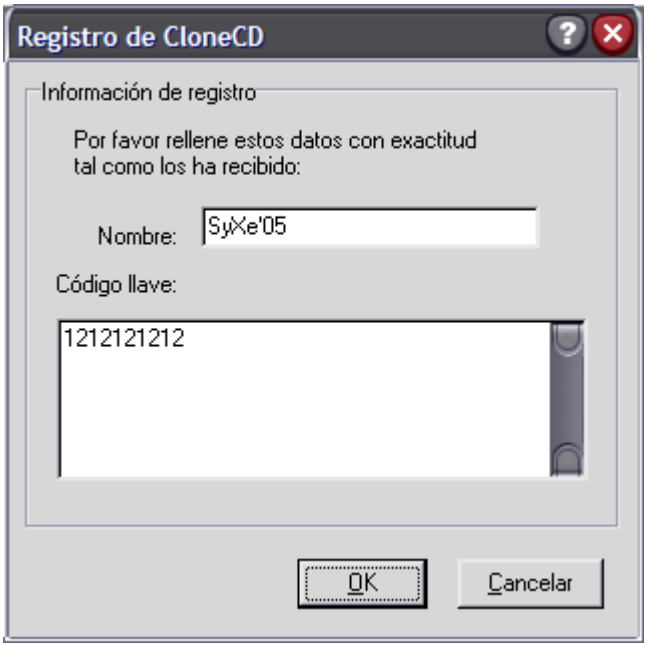
```
OllyDbg - ultimo_1.exe - [CPU - main thread, module ultimo_1]
File View Debug Plugins Options Window Help

00534C00   60              PUSHAD
00534C01   6A 04           PUSH 4
00534C03   68 D0070000     PUSH 7D0
00534C08   68 00800100     PUSH 18000
00534C0D   68 0000CF00     PUSH 0CF0000
00534C12   FF15 44454F00   CALL DWORD PTR DS:[4F4544]     kernel32.VirtualAlloc
00534C18   85C0            TEST EAX,EAX
00534C1A   6A 04           PUSH 4
00534C1C   68 00300000     PUSH 3000
00534C21   68 00800100     PUSH 18000
00534C26   68 0000CF00     PUSH 0CF0000
00534C2B   FF15 44454F00   CALL DWORD PTR DS:[4F4544]     kernel32.VirtualAlloc
00534C31   85C0            TEST EAX,EAX
00534C33   6A 00           PUSH 0
00534C35   6A 00           PUSH 0
00534C37   6A 03           PUSH 3
00534C39   6A 00           PUSH 0
00534C3B   6A 00           PUSH 0
00534C3D   68 00000080     PUSH 80000000
00534C42   68 C04C5300     PUSH ultimo_1.00534CC0          ASCII "final.bin"
00534C47   FF15 8C434F00   CALL DWORD PTR DS:[4F438C]     kernel32.CreateFileA
00534C4D   85C0            TEST EAX,EAX
00534C4F   6A 00           PUSH 0
00534C51   68 D04B5300     PUSH ultimo_1.00534BD0
00534C56   68 00800100     PUSH 18000
00534C5B   68 0000CF00     PUSH 0CF0000
00534C60   50              PUSH EAX
00534C61   FF15 E4444F00   CALL DWORD PTR DS:[4F44E4]     kernel32.ReadFile
00534C67   85C0            TEST EAX,EAX
00534C69   61              POPAD
00534C6A - E9 91C3ECFF     JMP ultimo_1.00401000
00534C6F   0000            ADD BYTE PTR DS:[EAX],AL
00534C71   0000            ADD BYTE PTR DS:[EAX],AL
```

Tened en cuenta que no lo puse en la página anterior porque no se si funcionaría y no quería llenar el tuto de más imágenes a no ser que sean definitivas y esta por supuesto, lo es, ;XDDD

Ahora que hemos obtenido un exe funcional y sin empaquetar podemos meterle mano, ya es el momento de atacar al Time-Trial y por supuesto a la nag inicial que encima trae un retardo creo que de 5 segundos y no mola naaaada Como siempre debo agradecer a los grandes maestros como Ricardo Narvaja, +NCR y es este caso especial a Saccopharynx del que he sacado la idea y el formato del injerto, lo siento tío es que era muy bueno ;)

Ahora sigamos, ya que estamos no nos vamos a detener aqui, pienso hacer un tuto de inicio a fin no uno de esos a medias que te quedas con la boca partía, es como ver una peli de la que quitán el final que suele ser lo más interesante, aqui esas cosas no pasan, coged palomitas que vamos a por la nag, jeje..

Primero centremos los objetivos, en este caso es claro y evidente pero hay casos en que no puede estar tan claro, la ventanita tiene 3 botones po lo que descartamos el típico ***MessageBox***, más ien podría ser un cuadro del tipo ***DialogBoxParamA*** o algo por el estilo, de momento desensamblaremos el exe en ***w32dasm*** y cargaremos el programa en Olly para ir echando un vistazo, bien pues antes lo digo y antes pasa, hay cambio de planes pues mirando entre las strings de wdasm me he topado con todas las cadenas referentes al registro del programa mediante serial, la de ***Thank you***, otra de ***Invalid*** no se qué, otra que es de una actualización.., en fin todas ellas están en una rutina que empieza en ***:41B6A4***, para estudiar bien la zona pondremos un ***BPX*** alli dede Olly qu tenemos cargado, luego vamos al cuadro de registro y ponemos lo que queramos:



Ahi esta puesto mi nombre y un serial de lo mas tipico, recuerdo los primeros tuts que lei de Estado+Porcino donde el ya utilizaba esta clave, todo un clasico del cracking, si yo fuera progamador y recibiera un key asi desde el ***EditBox*** es que vaya ni lo procesaba, jeje.. lo desestimaba ya de entrada ***'Invalid Key'*** seguramente haya un cracker ahi intentandolo petar;XD

Bueno se me van las neuronas a pasear de vez en cuando, retornemos al crackeo de CloneCD, aceptamos en el cuadro de diaçogo anterior y Olly rompe en el ***BPX*** que pusimos al inicio de la funcion, a partir de aqui es cuestion de tener paciencia y observar mucho lso registros, la pila y las estructuras de datos, tipicos bucles que cogen caracter a caracter del nombre y le suma x, etc., esto es muy imoprtante, cualquier detalle nos puede ayudar, entremos ya, comienza conn el tipico ***PUSH EBP*** de casi todas las funciones, no hay que tracear mucho para empezar a ver cosas:

```
:0041B6F3  6A 50          PUSH 50                          ; /Arg3 = 00000050
:0041B6F5  8D85 94FEFFFF  LEA EAX,DWORD PTR SS:[EBP-16C]   ; |
:0041B6FB  50             PUSH EAX                         ; |Arg2 = 0012F68C ASCII "SyXe'05"
:0041B6FC  8D95 E4FEFFFF  LEA EDX,DWORD PTR SS:[EBP-11C]   ; |
:0041B702  52             PUSH EDX                         ; |Arg1
:0041B703  E8 8C660000    CALL ultimo_1.00421D94           ; \ultimo_1.00421D94
```

se esta pusheando nuestro nombre con otros dos argumentos, aqui seguramente le aplicara mogollon de operaciones aritmeticas y/o logicas, de momento no nos interesa, entremos pero solo por ver que ahce sasi por encima no queremos un ***KeyGen***, asi a bote pronto puedo decir que ha copiado nuestro nombre en otra zona de memoria con todos los caracteres pasados a minusculas, le sigue esto:

```
:0041B70B  8D8D E4FEFFFF  LEA ECX,DWORD PTR SS:[EBP-11C]   ;En ECX a puna al nombre en minusculas
:0041B711  51             PUSH ECX                         ;Se lo pasa como argumento al sgute call
:0041B712  E8 41F20800    CALL ultimo_1.004AA958           ;calcula su longitud y la retorna en EAX
:0041B717  59             POP ECX                          ;recupera el nombre en minúsculas en ECX
:0041B718  83F8 03        CMP EAX,3                        ;mira si la longitud es igual a 3
:0041B71B  73 66          JNB SHORT ultimo_1.0041B783      ;y si no es inferior seguimos...
:0041B71D  68 3F080000    PUSH 83F                         ;caso de ser menor de 3 dígitos vamos a error
```

Un nombre menor de tres letras? como no nos llamemos Pi, o algo asi, que cosas mas raras hace esto, sigamos...

```
:0041B7D8  50              PUSH EAX                          ; |Arg2   ASCII = "1212121212"
:0041B7D9  8D95 34FFFFFF   LEA EDX,DWORD PTR SS:[EBP-CC]     ; |
:0041B7DF  52              PUSH EDX                          ; |Arg1 = 0012F72C
:0041B7E0  E8 63F9FFFF     CALL ultimo_1.0041B148            ; \ultimo_1.0041B148
```

Aqui veo que copia el serial a otra parte de la memoria y nada importante, sigamos...

atencion en esta parte:

```
:0041B7EE  51              PUSH ECX                          ; /Arg2
:0041B7EF  8D85 48FEFFFF   LEA EAX,DWORD PTR SS:[EBP-1B8]    ; |
:0041B7F5  50              PUSH EAX                          ; |Arg1
:0041B7F6  E8 A1FAFFFF     CALL ultimo_1.0041B29C            ; \ultimo_1.0041B29C
```

dentro del call 41B29C:

```
:0041B23E  8B55 FC         MOV EDX,DWORD PTR SS:[EBP-4]      ;en EDX dos primeros digitos serial
:0041B241  83FA 10         CMP EDX,10                        ;son un 10?
:0041B244  76 04           JBE SHORT ultimo_1.0041B24A       ;si es inferior o igual seguimos
:0041B246  33C0            XOR EAX,EAX                       ;activa flag de error
:0041B248  EB 48           JMP SHORT ultimo_1.0041B292       ;y nos saca del call
```

Esta mirando que los dos primeros digitos no sean superior a 10, si por ejemplo ponemos como serial 2345, lógicamente 23 > 10 y nos tira a los leones también, ahora hace otra cosa, empieza en un call llamémoslo padre pero va ocurriendo en calls hijas en una jerarquía creo que de dos o tres niveles más, pongo el árbol:

```
:0041B7F6  E8 A1FAFFFF   CALL ultimo_1.0041B29C
        :0041B2BB  E8 50FFFFFF   CALL ultimo_1.0041B210
                :0041B22B  E8 64FFFFFF   CALL ultimo_1.0041B194
                :0041B264  E8 2BFFFFFF   CALL ultimo_1.0041B194
```

Los dos que están al mismo nivel significa que son calls hermanas, en definitiva lo que hace aqui es lo siguiente: Recordáis que en la parte anterior miró los dos primeros dígitos y comprobó que no fueran superior a 10, esto tiene su lógica ahora, en esta serie de calls que he puesto lo que hace es entrar en un loop que iterará tantas veces como el número de los dos primeros dígitosle marque, en nuestro caso como pusimos 10xxxxxx iterará 10 veces, pero ojo que en cada iteración coge 4 dígitos del propio serial, con que en una pasada le falte algún dígito nos tirará para afuera, o sea que para un serial que comience por 10 como el nuestro deberá tener almenos 16x4 = 64 dígitos ya que contamos en hexadecimal, ademas a la salida del call anterior:

```
:0041B22B  E8 64FFFFFF   CALL ultimo_1.0041B194
```

Aqui tambien ademas se verifica que dejemos dos digitos mas al final del serial y que a su vez sea como paso con los dos primeros inferior o igual a 10 como maximo, o sea que el serial no puede acabar por ejemplo en 14 porque 14 > 10 y nos envia a error. Hay que tener en cuenta ademas que para todas estas cuentas lso dos primeros digitos no cuentan en el loop anterior del 4x1 o sea que tendra como minimo el siguiente numero de digitos suponiendo como es el caso de que empiece por 10.

[2 + (16x4) + 2] = [2 + 64 +2] = 68 digitos

luego se mete otra vez en el loop y vuele a ocurrir lo mismo, cuenta a 4x1 nuevamente en:

```
:0041B264  E8 2BFFFFFF   CALL ultimo_1.0041B194
```

ademas han de ser numeros y no letras, pongamos un 10 y 68 digitos mas, ya son: 64x2=136 dígitos

1012121212121212121212121212121212121212121212121212121212121212121212121111101515151515151515151515151
515151515151515151515151515151515151515151010

Este es el serial que me va saliendo para ***SyXe'05*** para otro nombre no se si valdra imagino que tambien, pero ahora hay un problema porque entra en una excepcion aqui:

```
:00423370   A1 B8454C00      MOV EAX,DWORD PTR DS:[4C45B8]        ;Se pone EAX=0...!!
:00423375   66:8B0C48        MOV CX,WORD PTR DS:[EAX+ECX*2]
```

esto ocurre si estamos traceando en ***Olly*** aun en el original, aunque el dumpeado caemos en la excepción con o sin ***Olly***, esta excepción ocurre dentro del call:

```
:0041B812   E8 8DE0FFFF    CALL ultimo_.004198A4
        :004198DF   E8 B8900000    CALL ultimo_.0042299C
```

Bueno quería sacar un serial válido pero con el tema de las excepciones este tendremos que parchear ;(no hay más remedio, pero esto no es problema ya en en la lista muerta vemos los mensajes de felicitación, de error, etc., y asi vemos cuales son los saltos que nos interesan y cuales no, resumire por encima el codigo interesante:

```
:0041B7F6   E8 A1FAFFFF     CALL ultimo_1.0041B29C        ;Debe retornar EAX=1
:0041B7FB   83C4 08         ADD ESP,8
:0041B7FE   85C0            TEST EAX,EAX
:0041B800   74 1C           JE SHORT ultimo_1.0041B81E    ;No debe saltar
:0041B802   6A 00           PUSH 0                                  ; /Arg3 = 00000000
:0041B804   8D95 48FEFFFF   LEA EDX,DWORD PTR SS:[EBP-1B8]          ; |
:0041B80A   52              PUSH EDX                                ; |Arg2
:0041B80B   8D8D 94FEFFFF   LEA ECX,DWORD PTR SS:[EBP-16C]          ; |
:0041B811   51              PUSH ECX                                ; |Arg1
:0041B812   E8 8DE0FFFF     CALL ultimo_1.004198A4                  ; \ultimo_1.004198A4
:0041B817   83C4 0C         ADD ESP,0C
:0041B81A   8BD8            MOV EBX,EAX                   ;Debe ser EAX=2 aqui..!!
:0041B81C   EB 02           JMP SHORT ultimo_1.0041B820
:0041B81E   33DB            XOR EBX,EBX
:0041B820   B2 01           MOV DL,1
:0041B822   8B86 E4020000   MOV EAX,DWORD PTR DS:[ESI+2E4]
:0041B828   8B08            MOV ECX,DWORD PTR DS:[EAX]
:0041B82A   FF51 5C         CALL DWORD PTR DS:[ECX+5C]
:0041B82D   33D2            XOR EDX,EDX
:0041B82F   A1 80FF4D00     MOV EAX,DWORD PTR DS:[_RegisterForm]
:0041B834   E8 73F00500     CALL ultimo_1.0047A8AC
:0041B839   8B0D 80FF4D00   MOV ECX,DWORD PTR DS:[_RegisterForm]
:0041B83F   8B81 D0020000   MOV EAX,DWORD PTR DS:[ECX+2D0]
:0041B845   B2 01           MOV DL,1
:0041B847   8B08            MOV ECX,DWORD PTR DS:[EAX]
:0041B849   FF51 5C         CALL DWORD PTR DS:[ECX+5C]
:0041B84C   80EB 01         SUB BL,1                      ;  Switch (cases 0..4)
:0041B84F   72 18           JB SHORT ultimo_1.0041B869    ;Salta a Invalid Key..!!
:0041B851   0F84 98000000   JE ultimo_1.0041B8EF          ;Key EXPIRED..!!(
:0041B857   80EB 02         SUB BL,2
:0041B85A   0F84 00010000   JE ultimo_1.0041B960          ;El Código es una actualización..!!
:0041B860   FECB            DEC BL
:0041B862   74 3F           JE SHORT ultimo_1.0041B8A3    ;El código es tempora y encima EXPIRADOl..!!
:0041B864   E9 8F010000     JMP ultimo_1.0041B9F8         ;A Thank You...!!!
```

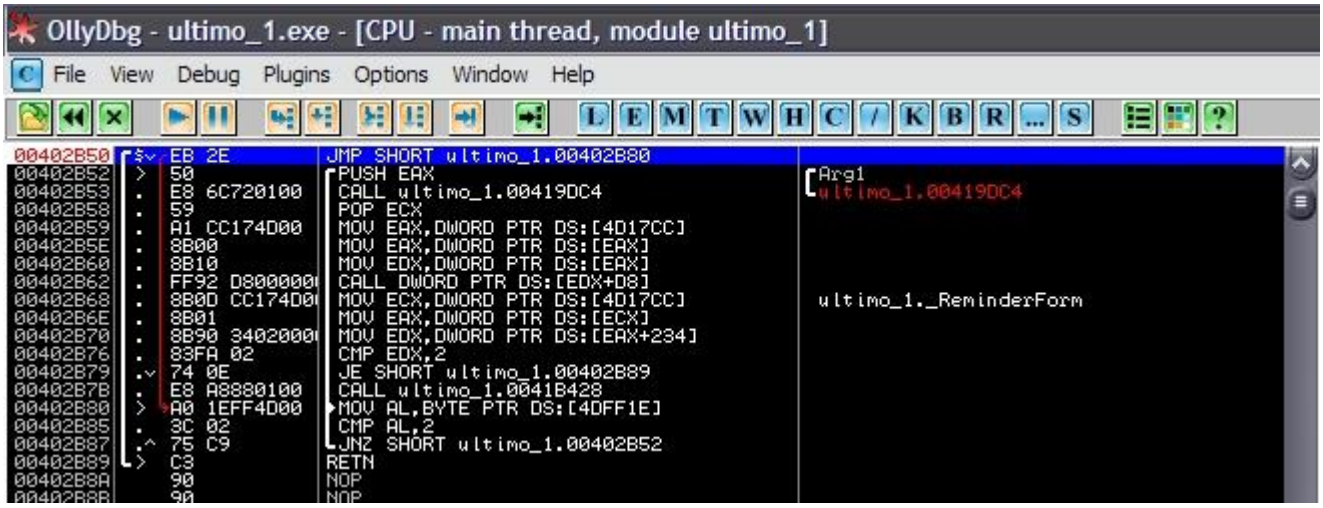
El problema es que como saltan las expciones deberíamos trabajar más de la cuenta para registrarnos por aqui, asi que lo que haremos será una maniobra rápida, es decir atacaremos por otro lado, venga todos a recargar, ***Ctrl+F2*** y ***BPX*** en ***EnableWindow***, ***F9*** y a esperar la nag.. rompe ***Olly***, ***Ctrl+F9*** para retornar a:

```
:0047FD5D   50              PUSH EAX                                ; /Enable
:0047FD5E   8B83 4C010000   MOV EAX,DWORD PTR DS:[EBX+14C]          ; |
:0047FD64   50              PUSH EAX                                ; |hWnd
:0047FD65   E8 42AA0300     CALL <JMP.&user32.EnableWindow>         ; \EnableWindow
```

Ahí está la llamada con sus argumentos, sigamos a partir de aqui con F8 hasta que aparezca la nag en pantalla, antes quitemos el BPX de la api para que no nos moleste, no hay que tracear mucho y llegaremos a:

:00402D88 . E8 C3FDFFFF CALL ultimo_1.00402B50

Aqui sale la nag pero también parte del programa principal, ponemos un BPX con F2 sobre el call y a reiniciar todos, F9 y caemos encima del call, entramos con F7 y a tracar con F8 como locos ;XDD pero qué digo mirad un momento, esto merece una foto, jeje..



Estamos en :402B50 donde inicia el call anterior, el salto nos lleva a un:

:00402B80 A0 1EFF4D00 MOV AL,BYTE PTR DS:[4DFF1E]
:00402B85 3C 02 CMP AL,2
:00402B87 75 C9 JNZ SHORT ultimo_1.00402B52 ;Si no salta entramos sin la nag inicial

verifica si el flag ***[4DFF1E]*** vale ***2***, si no es así salta a ***:402B52*** y allí pinta la nag...!!! es decir que la aparición de la nag en pantalla está condicionada al valor de ***[4DFF1E]*** no lo sentís? esta es una variable Global del Proceso que puede ser utilizada desde cualquier procedimiento esté en el nivel que esté, da igual porque es ***GLOBAL***, muy comúnmente utilizada para estos fines de Registrados / no Registrados, etc.. he aqui la importancia de esta, podríamos sin más nopear el salto en ***:402B87*** y no ver la nag pero vamos a ser más curiosos, pondremos un ***BPM*** en dicha variable a ver donde nos lleva, a ver si descubrimos donde se inicializa y alli atacamos con mayor contundencia, venga a poner el ***BPM On Memory Acces*** pero ya y reiniciamos ***Ctrl+F2***, luego ***F9*** y... NO PARA en nuestro BPM ¿?

Vamos a w32dasm y busquemos las referencias a ***[4DFF1E]*** a ver que encontramos cosas interesantes como esta:

:0040773B 8A0D1EFF4D00 mov cl, byte ptr [004DFF1E] ***;Chequea el FLAG..!!***
:00407741 80F902 cmp cl, 02 ***;mira si vale 2***
:00407744 7416 je 0040775C ***;si es asi se salta la limitacion, jeje..***

* Possible Reference to String Resource ID=02162: "CloneCD is not registered! Writing will be done with 2x writ"
|
:00407746 6872080000 push 00000872

Parece que estamos persiguiendo la variable correcta, como se ve de ella dependen ciertas limitaciones que trae el programa, veamos donde se modifica:

:0041970E 881D1EFF4D00 mov byte ptr [004DFF1E], bl

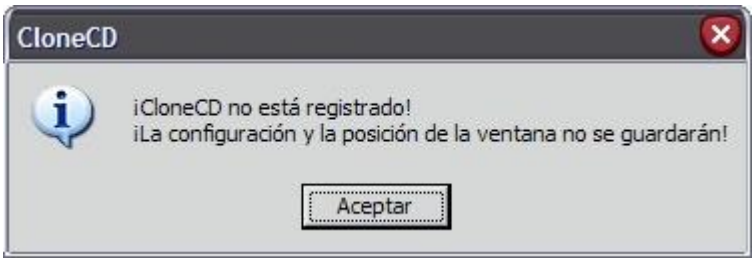
Sólo aparece esta, será cuando nos registramos que viene aqui y le pone un mágico 2? pero por supuesto aqui no paramos, se me ocurre otra idea, cuando es la primera vez que para? pues alli cambiar:

:00402B80 A0 1EFF4D00 MOV AL,BYTE PTR DS:[4DFF1E]
:00402B85 3C 02 CMP AL,2
:00402B87 75 C9 JNZ SHORT ultimo_1.00402B52 ;Si no salta entramos sin la nag inicial

POR:

```
:00402B80    C605 1EFF4D00>  MOV BYTE PTR DS:[4DFF1E],2
:00402B87    B0 02           MOV AL,2
```

Con este cambio entramos sin la nag pero seguimos sin estar registrados como vemos en el About y al salir:



Pero tenemos otro mensaje al que seguirle la pista no? a todo hay que sacarle su parte buena que creedme la tiene:

```
:004025D6   A1 F81F4D00      MOV EAX,DWORD PTR DS:[4D1FF8]
:004025DB   85C0             TEST EAX,EAX
:004025DD   0F84 1E010000    JE ultimo_.00402701
:004025E3   8A90 644B0000    MOV DL,BYTE PTR DS:[EAX+4B64]
:004025E9   84D2             TEST DL,DL
:004025EB   75 18            JNZ SHORT ultimo_.00402605    ;Bonito salto..!!
:004025ED   68 6F080000      PUSH 86F
:004025F2   E8 59F60100      CALL ultimo_.00421C50
:004025F7   59               POP ECX
:004025F8   50               PUSH EAX
:004025F9   6A 00            PUSH 0
:004025FB   E8 C0A90100      CALL ultimo_.0041CFC0         ;Aqui saca el mensaje por pantalla..!!
:00402600   83C4 08          ADD ESP,8
:00402603   EB 19            JMP SHORT ultimo_.0040261E
```

Esta es la parte donde se decide si se va a mostrar o no el mensaje, yo destacaría las líneas:

```
:004025E3   8A90 644B0000    MOV DL,BYTE PTR DS:[EAX+4B64]
:004025E9   84D2             TEST DL,DL
:004025EB   75 18            JNZ SHORT ultimo_.00402605
```

Se está verificando el valor de ***[EAX+4B64]*** que haciendo un ***Follow in Dump -> Memory Address*** descubrimos que representa a ***[4D6B60]*** y tiene el byte bajo que es el que se está verificando puesto a 0, pues ya tenemos otra dirección, ***BPM*** en ella y a reinicir se ha dicho a ver donde nos lleva, pues a ninguna parte, para una única vez y no es para escribir en el flag, estamos en las mismas que antes, aún poniéndolo a un valor distinto de 0 lo único que conseguimos es salir sin el molesto cartel, pero en About dice que no estamos registrados, pero bueno igual cambiamos la instrucción:

```
:004025EB   7518     JNZ SHORT ultimo_.00402605    POR:
:004025EB   EB18     JMP SHORT ultimo_.00402605
```

Y ya no vemos el mensaje de salida más.

Esto está muy bien pero el haber localizado el flag maestro nos hubiera ayudado mucho ya que él solito nos hubiera hecho todo este trabajo, nosotros le vamos pillando variables auxiliares y de esto nos vamos beneficiando, no está mal pero podría estar mejor, si probamos ahora el archivo no veremos más nags, no tengo grabadora y no puedo probar las grabaciones pero intuyo que irán bien, si no es así pido porfavor que se me avise para repararlo, si ponemos un ***BPX*** en ***RegOpenKeyA*** y ***RegOpenKeyExA*** veremos como va accediendo al registro y a por qué valores, nos llama mucho la atención la clave:

[HKEY_CURRENT_USER\SOFTWARE\Elaborate Bytes\CloneCD\User\C32AC277]

aqui en caso de localizar la clave busca los valores ***UserName***, ***UserKeyS***, ***UserKeyR*** y ***UserKeyCRC***, pero açun poniendolos alli junto con la clave no conseguimos nada por lo que desestimo esta via ya, ademas lo unico que queda por eliminar es un ***¡NO REGISTRADO!*** de la ventanita ***About***, el resto es historia (creo vaya).

Repasemos los cambios que hemos hecho despues de desempakar:

1.- Para que no nos cierre el programa:

```
:004012F8   7511        JNZ SHORT ultimo.0040130B        POR:
:004012F8   7511        JNZ SHORT ultimo.0040130B
```

2.- Para no ver la nag:

```
:00402B87  75C9         JNZ SHORT ultimo.00402B52        POR:
:00402B87  9090         NOP's
```

3.- Para no ver el mensaje de salida:

```
:004025EB   7518        JNZ SHORT ultimo.00402605        POR:
:004025EB   EB18        JMP SHORT ultimo.00402605
```

Con esto no vemos nags ni nadas pero no nos guarda la configuracion personal que pongamos y la cadena del About es lo que nos queda por solventar.

Siguiendo las cadenas con ***w32dasm*** vemos ques en este call:

```
:0040E0BE   E8 8D3B0100        CALL ultimo3.00421C50
:0040E0C3    59                POP ECX
```

se pone en EAX la cadena ***¡NO REGISTRADO!***, si entramos en el call vemos que recoge el puntero en una asignación con desplazamiento como esta:

```
:00421C68  | 8B0495 6C9D4E>     MOV EAX,DWORD PTR DS:[EDX*4+4E9D6C]
```

en este caso ***[EDX*4+4E9D6C]*** representa ***[4EBDB8]*** que tiene el valor ***D156C8***, si vamos allí vemos la cadena buscada ***¡NO REGISTRADO!*** podríamos montarnos algo para cambiarla por nuestro nombre pero esto no evitaría que no se salve la configuración que como sabemos no la guarda, podemos includo eliminarla cambiando:

```
:0040E0F6   74 31        JE SHORT ultimo3.0040E129        POR:
:0040E0F6   90 90        NOP'S
```

Aunque esto no hace que veamos nuetro nombre pero tampoco la molesta cadena. Ahora solo quedaría ver porqué no guarda nuestra configuración, esto ya si que lo dejo en vuestras manos, estoy muuuuuuuuy cansado de este programa ya, voy a pasar a otro que ya toca, estoy con este mismo muchas horas ya y eso cansa bastante, afortuntadamente el trabajo duro lo pudimos realizar, jeje.. costó pero se acabó que es lo que importa.

Con esto damos por finalizado el tuto, he intentado mostrar la manera en la que nos podemos enfrentar a un progama de este tipo, empaquetado con ***ASProtect*** que crea otro archivo también empaquetado en tiempo de ejecución que es la verdadera aplicación y a la que hemos atacado, más que nada intento enseñar los problemas que nos pueden ir saliendo y como no, posibles soluciones, unas válidas y otras no, pero para eso estamos para aprender ;XDDD

Para cualquier duda, pregunta, sugerencia, crítica o lo que queráis sentíos libres de escribirme a:

syxe00@yahoo.es
syxe00@hotmail.com

Por supuesto todos los mensajes serán contestados. Hasta la próxima